Software Design Specification

for

Campus Food Ordering and Management System

Version <2.0>

Group No.: 1

< Hassan Abdelhaleem Hassan Basheir >	**< 251UC2506T >**
< Rashad Ali Qaid Sofan >	**< 251UC2505G >**
< Ahmad Aljammal >	**< 251UC2500R >**
< Mohammed A. M. Alakklouk >	**< 251UC250CR >**

Date: 02/06/2026

Contents

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
SRS in Part 1(as Ver 1.0)	Hassan Basheir, Rashad Sofan, Ahmad Aljammal, Mohammed Alakklouk	We kicked things off by defining the project scope, identifying the main actors, and getting the basic use cases down on paper.	15/04/2026
SDS in Part 2(as Ver 2.0.X)	Hassan Basheir & Rashad Sofan	Started working on the design specifics, focusing on the system architecture and the first batch of activity diagrams for student and vendor actions.	10/05/2026
SDS in Part 2(as Ver 2.0.X)	Ahmad Aljammal & Mohammed Alakklouk	Took over the technical data design, building the class diagrams and mapping out how the database would handle orders and payments.	22/05/2026
SDS in Part 2(as Ver 2.0.X)	Hassan Basheir, Rashad Sofan, Ahmad Aljammal, Mohammed Alakklouk	Finished up the remaining activity diagrams (2.1.8 to 2.1.15), finalized the deployment layout, and cleaned up the whole document for submission.	30/05/2026
*System Documentation in Part 3 (as Ver 3.0)	Ahmad Aljammal & Mohammed Alakklouk	Finalized the technical documentation, making sure all the setup instructions and system summaries were clear and accurate.	01/06/2026
Draft Type and Number	Hassan Basheir, Rashad Sofan, Ahmad Aljammal, Mohammed Alakklouk	This is the final polished version (Draft #4) after we went through and fixed all the small errors and formatting issues.	02/06/2026

1 System Overview

1.1 Description

The Campus Food Ordering and Management System is a centralized digital platform designed to digitize and streamline food purchasing, vendor management, and queue control processes within the university campus. By integrating pre-ordering, scheduling, real-time tracking, and payment functionalities, the system addresses inefficiencies in traditional campus dining operations. The system allows students to browse vendor menus, add items to a digital shopping cart, schedule orders with specific pickup times, complete secure payments, and track order status and queue position in real-time. Vendors can manage their menus, update inventory availability, process incoming orders, and manage digital queue tickets. Administrators oversee user and vendor accounts, configure system settings, monitor campus-wide dining analytics, and generate compliance and performance reports. The platform integrates notification services, role-based access control, and a rating and review mechanism to ensure a transparent, efficient, and user-friendly dining experience on campus.

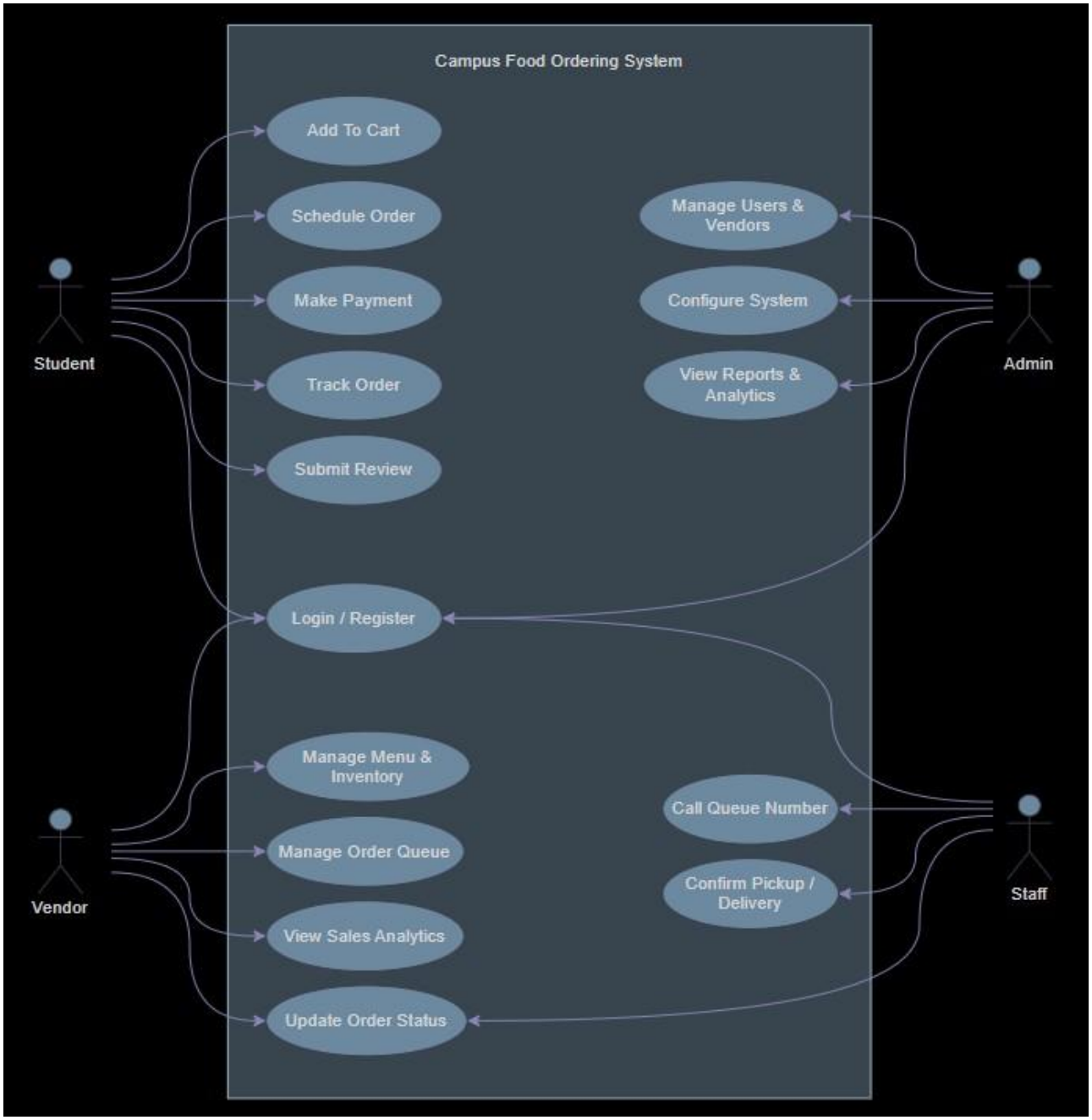
1.2 Actors

Actor	Use Cases
Student	Register/Login, Browse Menu & Add to Cart, Schedule Order & Select Pick-up Time, Process Payment, Track Order Status & Queue Position, Submit Ratings & Reviews
Vendor	Register/Login, Manage Food Menu & Inventory, View & Update Order Status, Manage Digital Queue, View Sales & Performance Analytics
Admin	Manage User & Vendor Accounts, Configure System Settings & Operating Hours, Monitor Campus-wide Analytics & Reports, Handle Support/Disputes, Manage Payment Gateway Integration
Staff	A staff member responsible for handling order fulfillment operations, including managing the order queue, updating order statuses (e.g., preparing, ready), calling queue numbers, and confirming order pickup.

1.3 Assumptions and Dependencies

- **Assumptions:**
    - All students, vendors, and staff have access to a smartphone or computer with a stable internet connection.
    - Vendors will maintain accurate and up to date menu items, pricing, and availability statuses.
    - The campus network infrastructure supports reliable communication between the application server and external services
    - Peak usage periods will primarily occur during standard campus meal times (e.g., 11:30 AM–2:00 PM, 5:00 PM–7:00 PM).
- **Dependencies:**
    - Third-party payment gateway (e.g. or TnG e-wallet) for secure transaction processing.
    - External notification services.
    - Campus authentication system.
    - Cloud hosting provider for scalable deployment and database backups.

1.4 Use Case Diagram

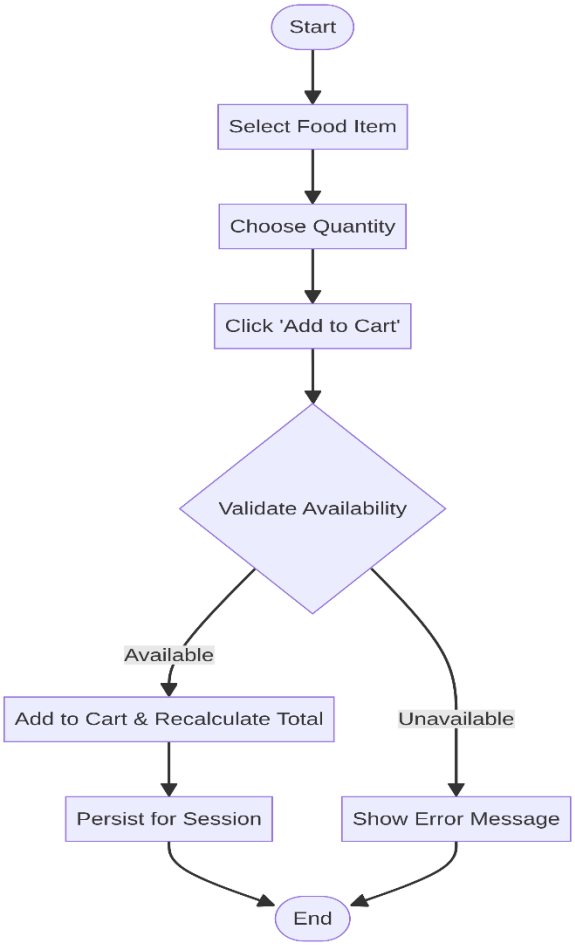


2 Activity Diagrams

2.1 Activity Diagrams

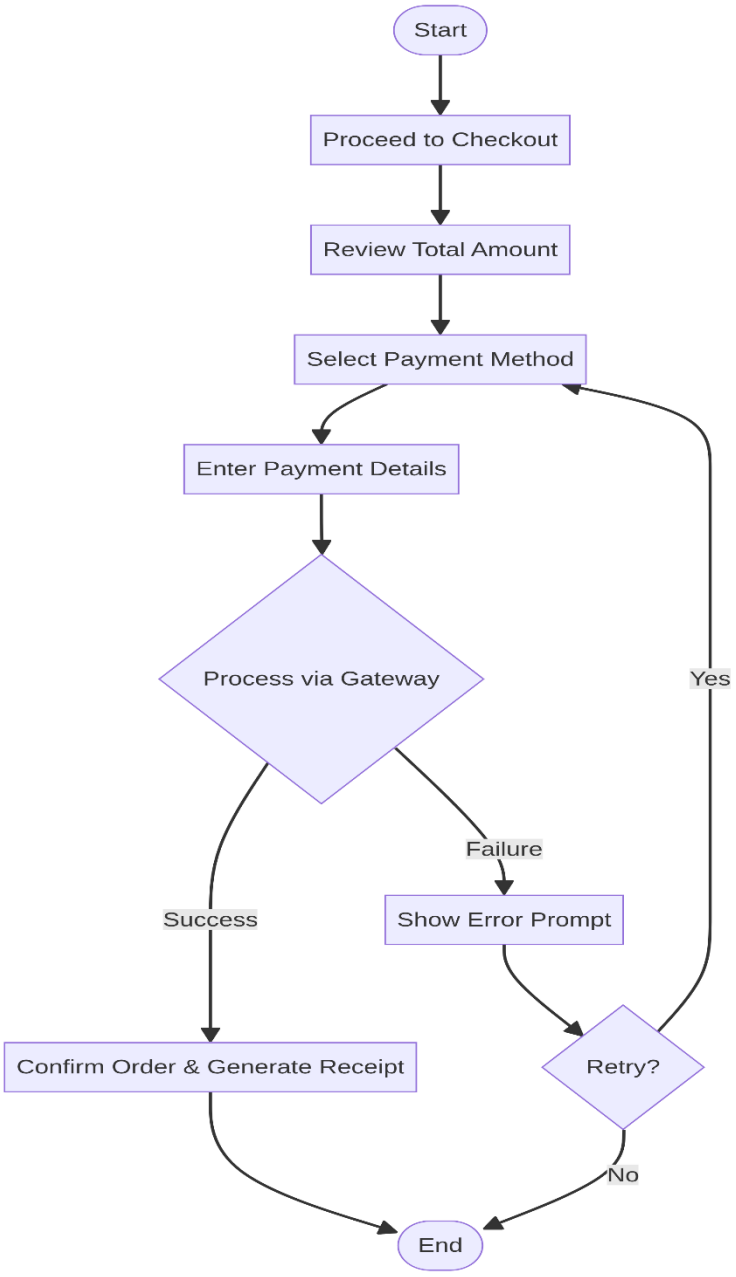
2.1.1 Add to Cart (Student)

The student selects a food item from the vendor menu, chooses a quantity, and clicks "Add to Cart". The system validates item availability. If available, the item is added to the cart and the total price is recalculated. If unavailable, an error message is shown. The cart persists for the session.



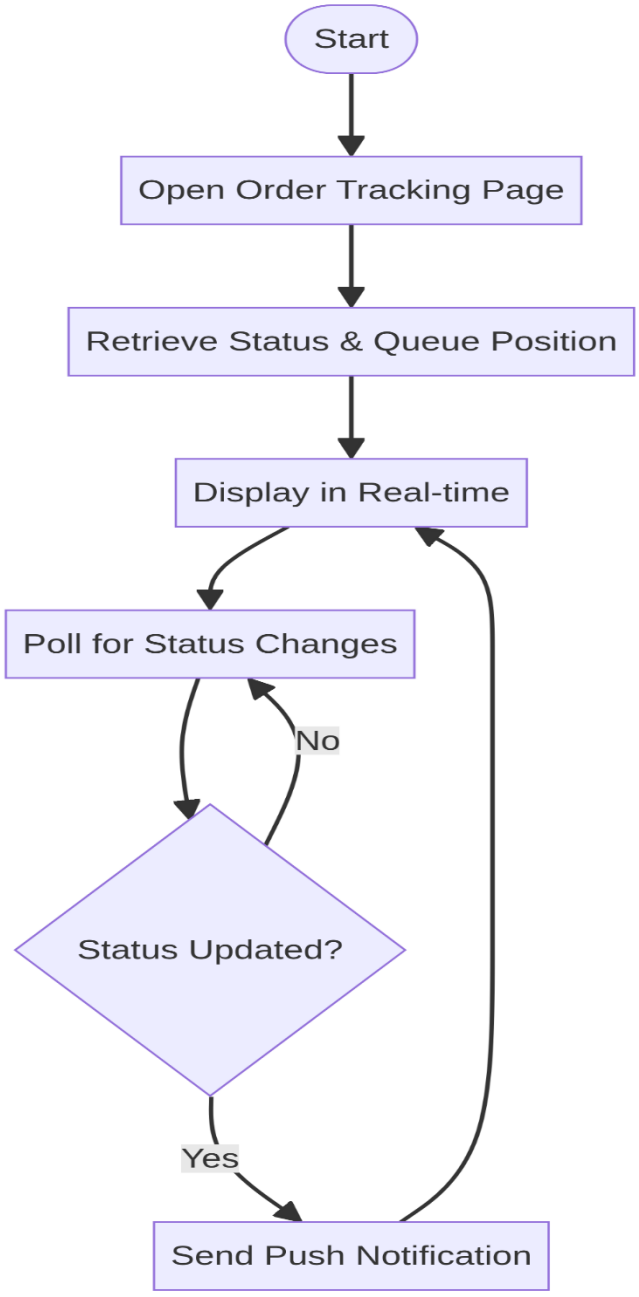
2.1.2 Make Payment (Student)

The student proceeds to checkout, reviews the total amount, selects a payment method (TnG e-wallet, credit/debit card, or online banking), and enters payment details. The system processes the payment via the external gateway. On success, the order is confirmed and a receipt is generated. On failure, an error prompt allows the student to retry.



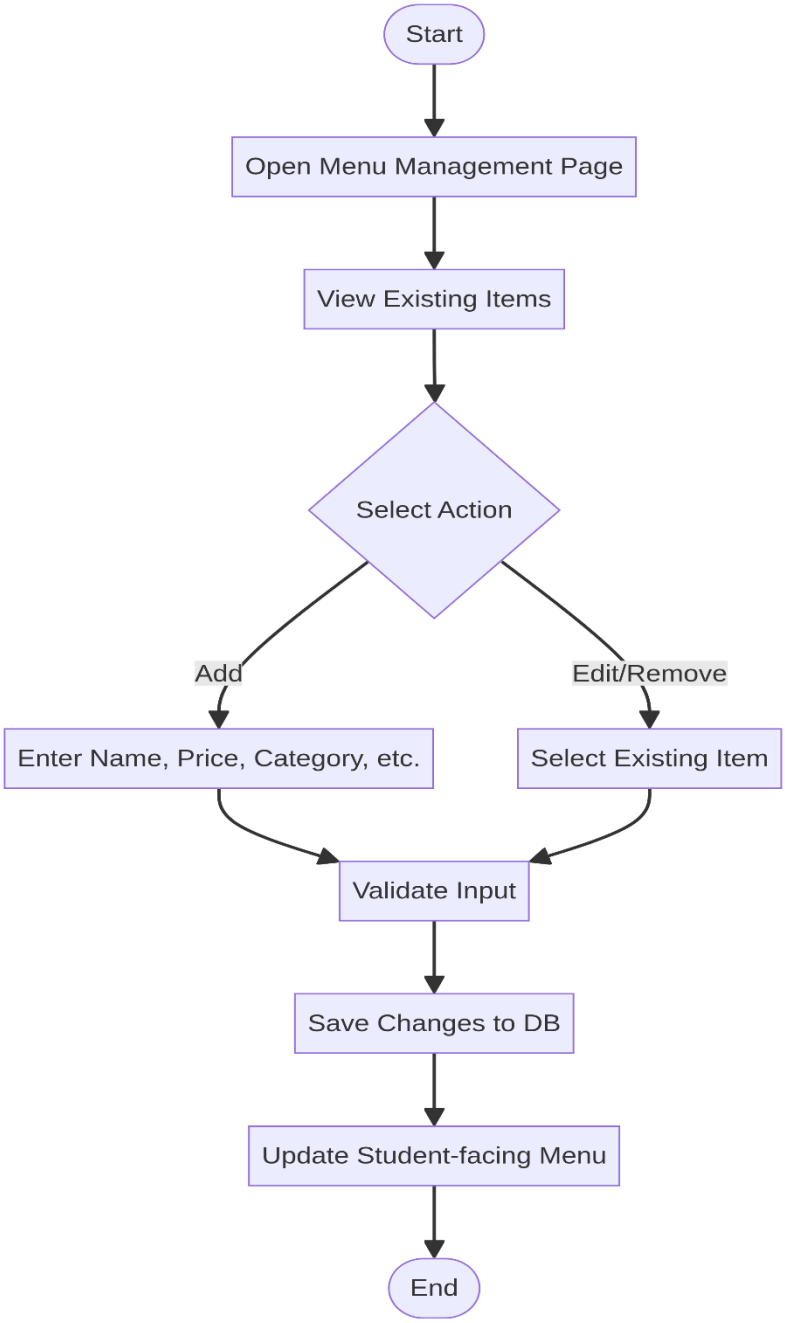
2.1.3 Track Order (Student)

After placing an order, the student opens the order tracking page. The system retrieves the current order status and queue position from the database and displays it in real-time. The system polls for status changes and sends a push notification to the student whenever the order status updates (e.g., Confirmed, Preparing, Ready).



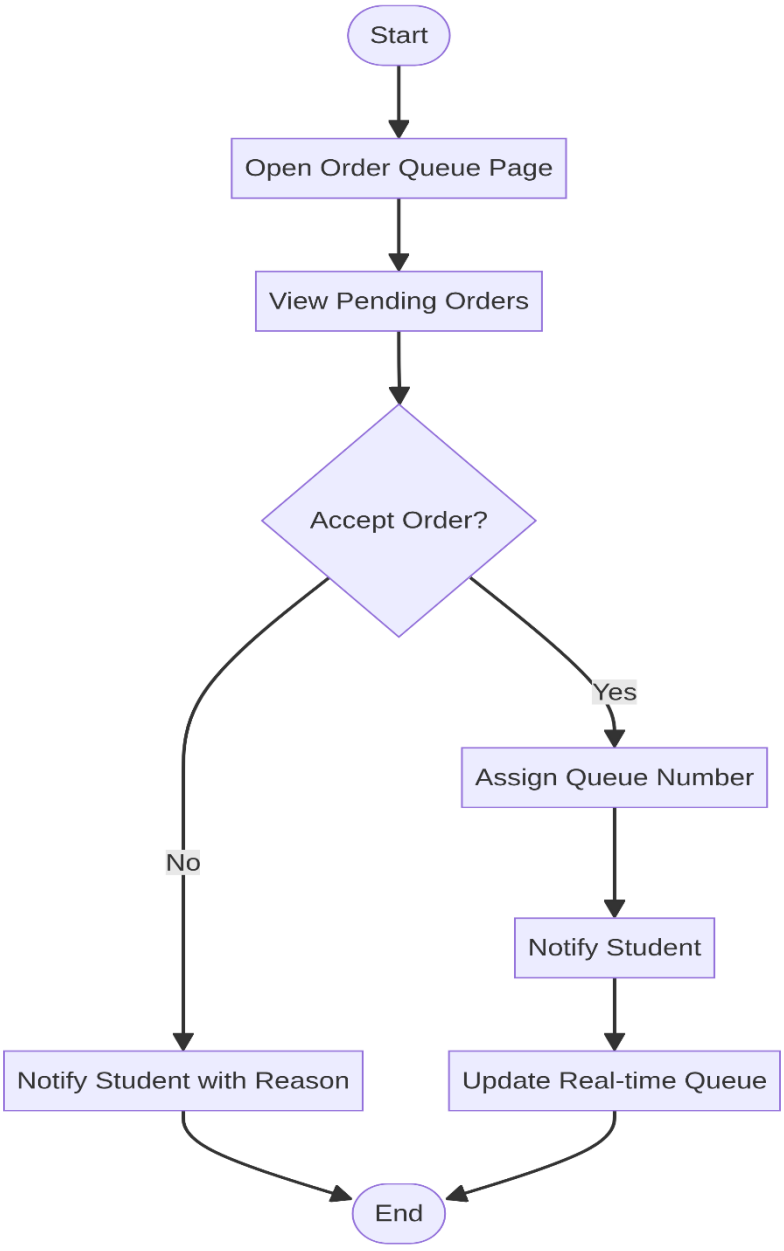
2.1.4 Manage Menu & Inventory (Vendor)

The vendor opens the Menu Management page and views all existing menu items. The vendor can add a new item by entering its name, price, category, preparation time, and uploading an image. Existing items can be edited or removed. The system validates all inputs and saves changes, which are immediately reflected in the student-facing menu view.



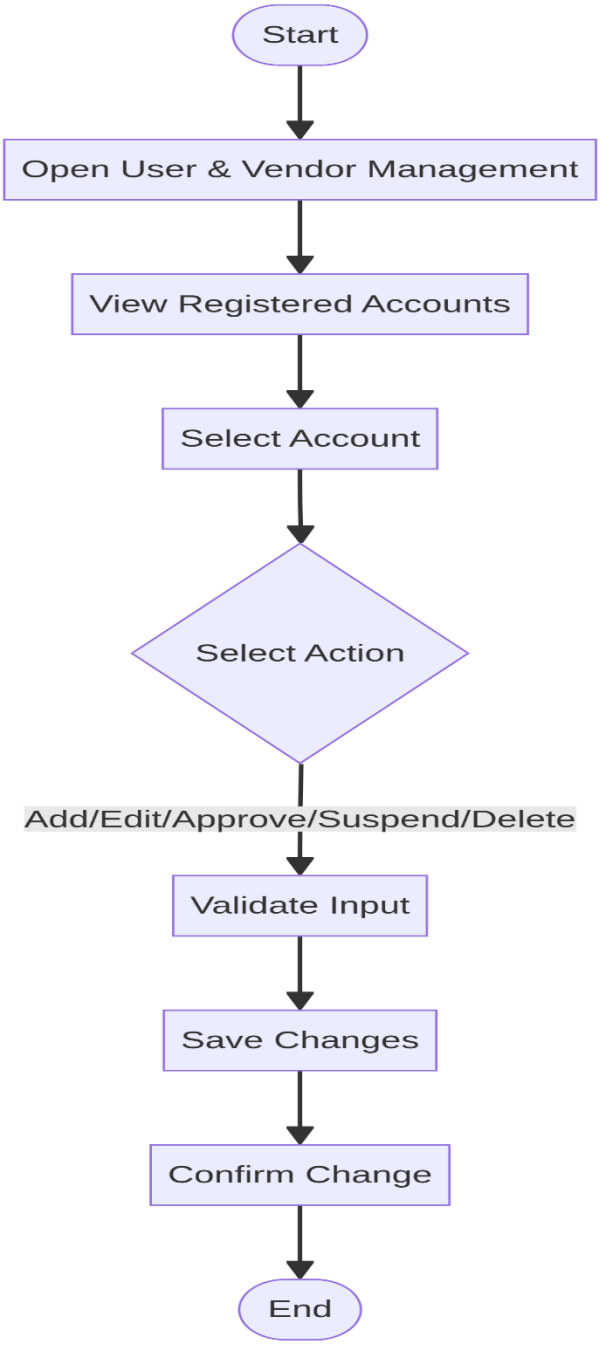
2.1.5 Manage Order Queue (Vendor)

The vendor opens the Order Queue page and views all pending incoming orders. The vendor can accept or reject each order. Upon acceptance, a queue number is assigned to the order and the student is notified. Upon rejection, the student is notified with a reason. The system updates the queue in real-time and reflects position changes to all active tracking sessions.



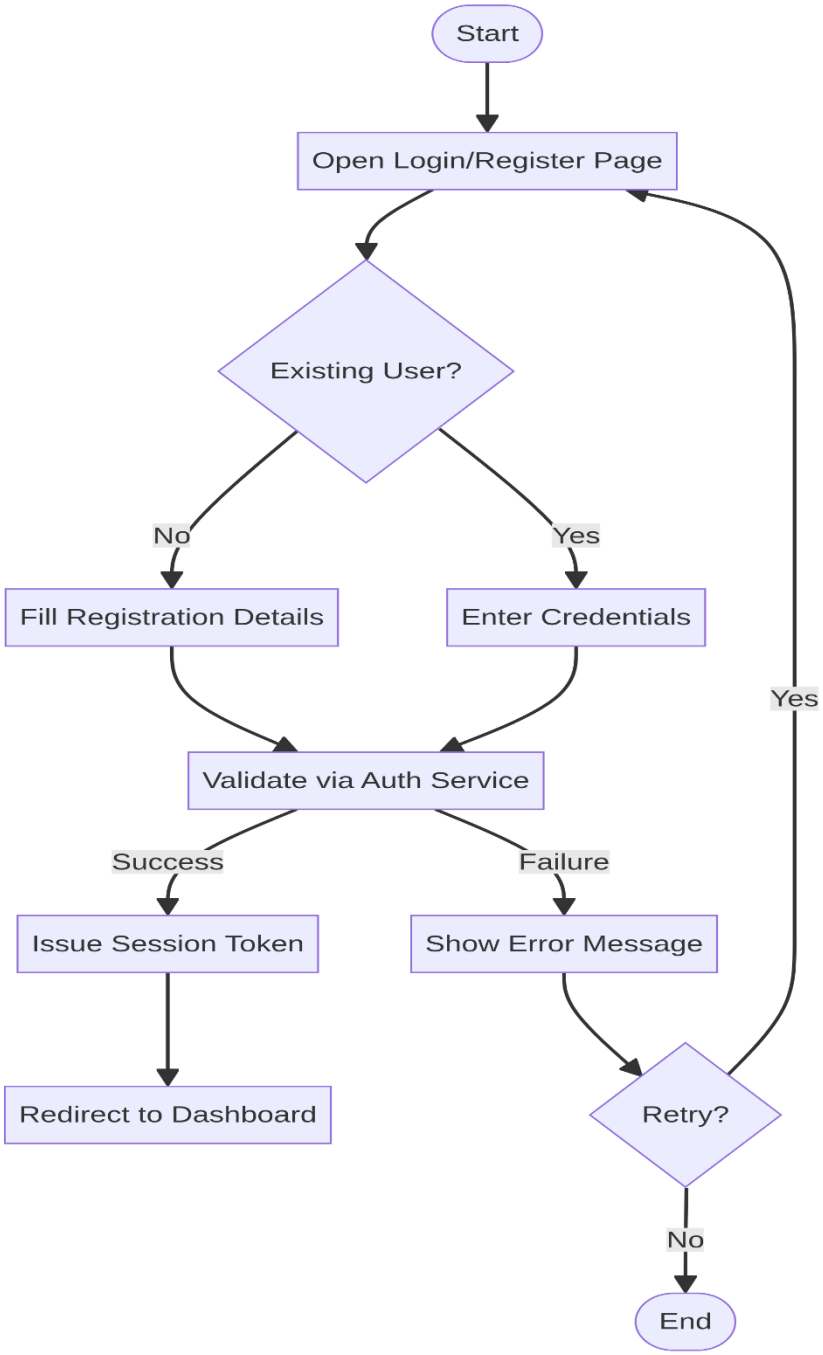
2.1.6 Manage Users & Vendors (Admin)

The administrator opens the User & Vendor Management page and views all registered accounts. The admin can select any account to add, edit, approve, suspend, or delete it. The system validates input before saving, then confirms the change. Restricted to admin users only via role-based access control.



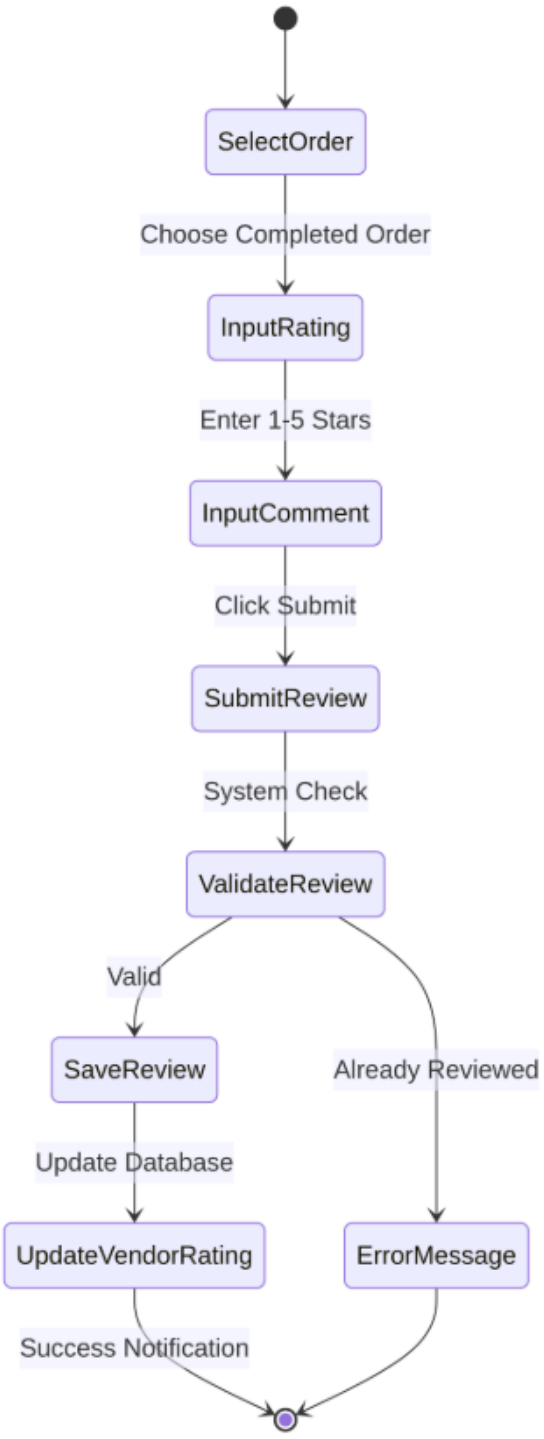
2.1.7 Login / Register (All Actors)

Any user (Student, Vendor, Admin, or Staff) opens the Login/Register page. New users fill in registration details; existing users enter their credentials. The authentication service validates the input against the database. On success, a session token is issued and the user is redirected to their role-specific dashboard. On failure, an error message is shown with an option to retry.



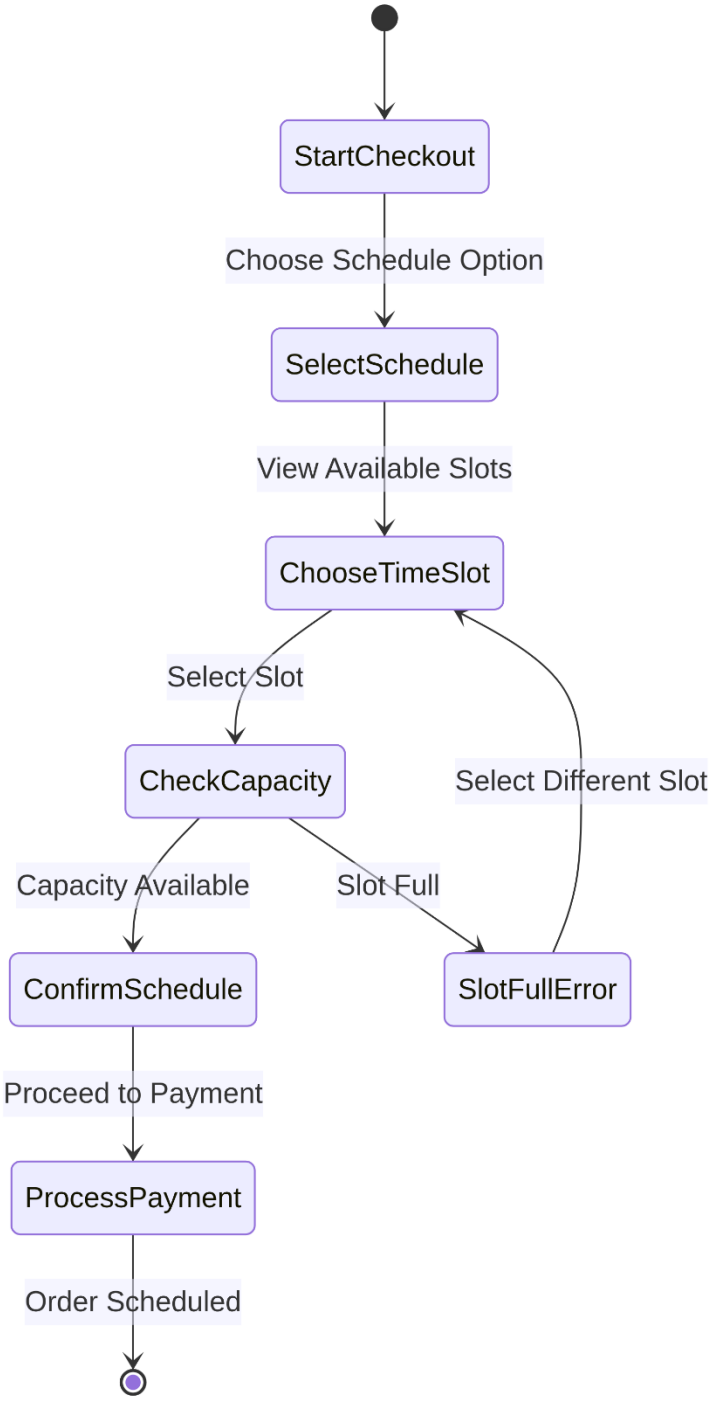
2.1.8 Submit Review (Student)

The student selects a completed order from their history and chooses to submit a review. The system prompts for a numeric rating (1-5 stars) and an optional text comment. Upon submission, the system validates that a review hasn't already been submitted for this order, saves the data, and automatically updates the vendor's average rating.



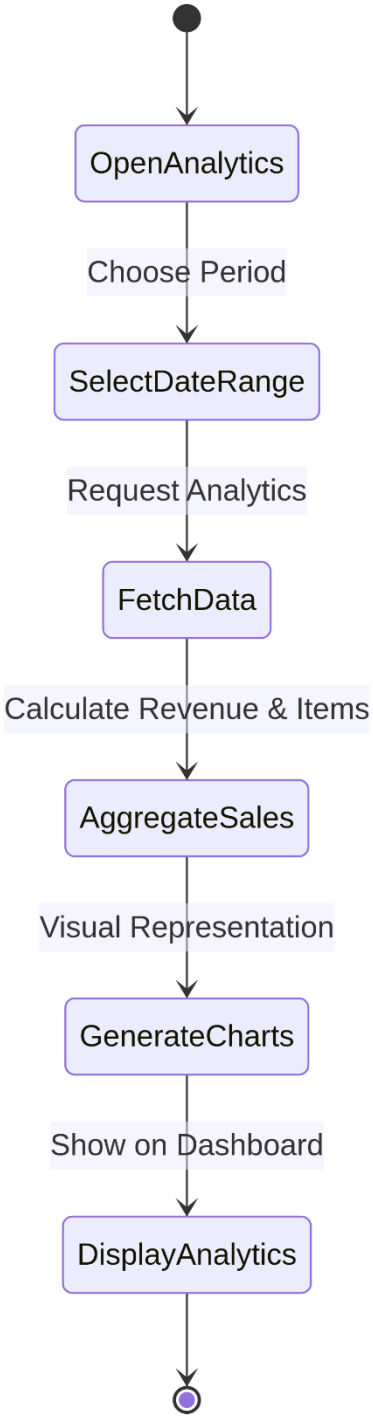
2.1.9 Schedule Order (Student)

During the checkout process, the student can choose to schedule their order for a future time slot instead of immediate preparation. The system displays available time slots based on vendor operating hours and current capacity. Once a slot is selected and payment is processed, the order is flagged as "Scheduled" and will only appear in the vendor's active queue at the appropriate time.



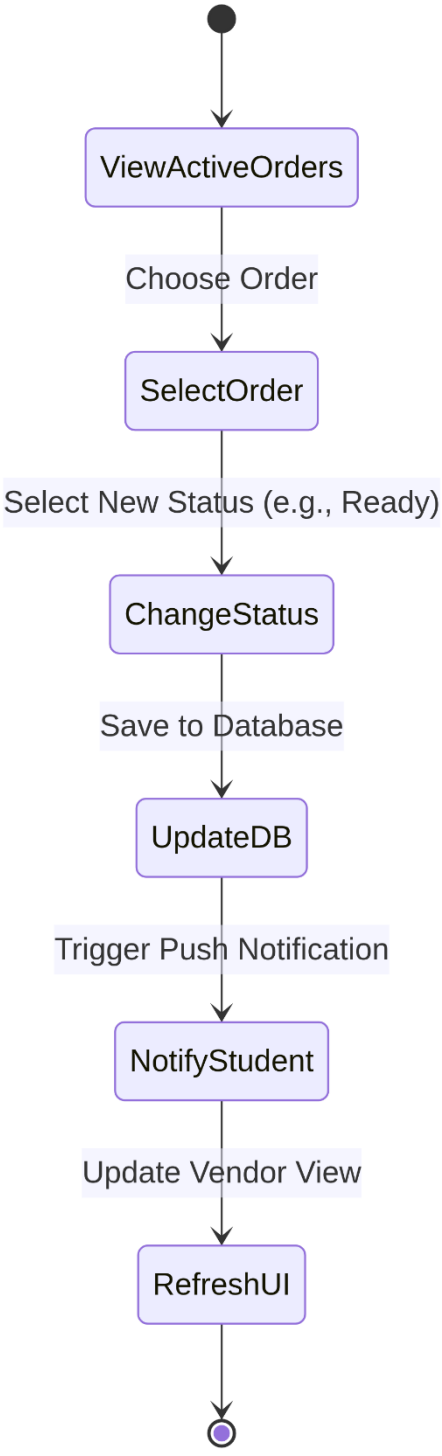
2.1.10 View Sales Analytics (Vendor)

The vendor accesses the Analytics dashboard to monitor business performance. They can select specific date ranges to view total revenue, most popular items, and peak ordering hours. The system aggregates data from the database and generates visual charts and tables to help the vendor make informed inventory and staffing decisions.



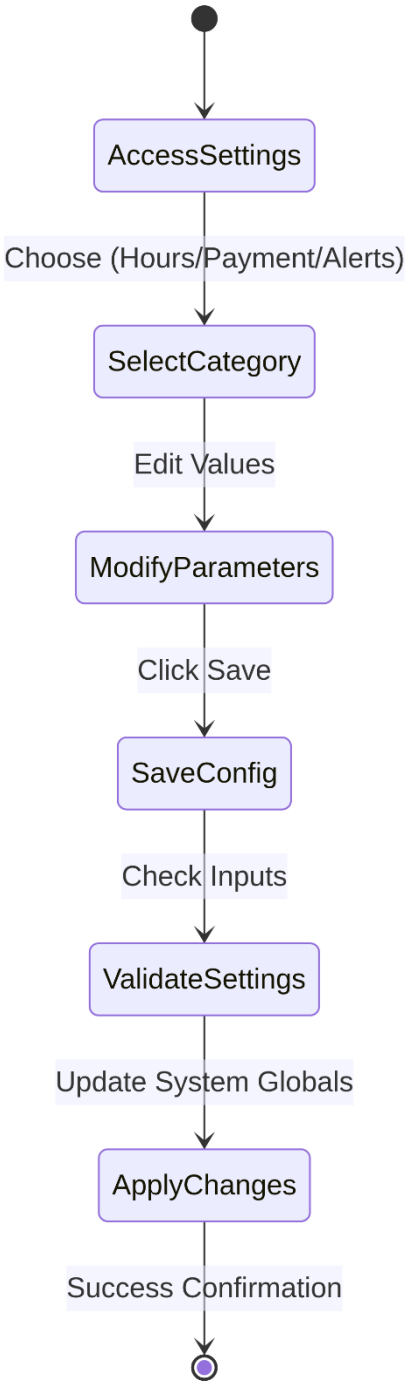
2.1.11 Update Order Status (Vendor)

the vendor progresses with an order, they manually update its status in the system. Transitions include moving an order from "Preparing" to "Ready". Each status change triggers an immediate push notification to the student's device and updates the real-time tracking interface to ensure the student is informed of their order's progress.



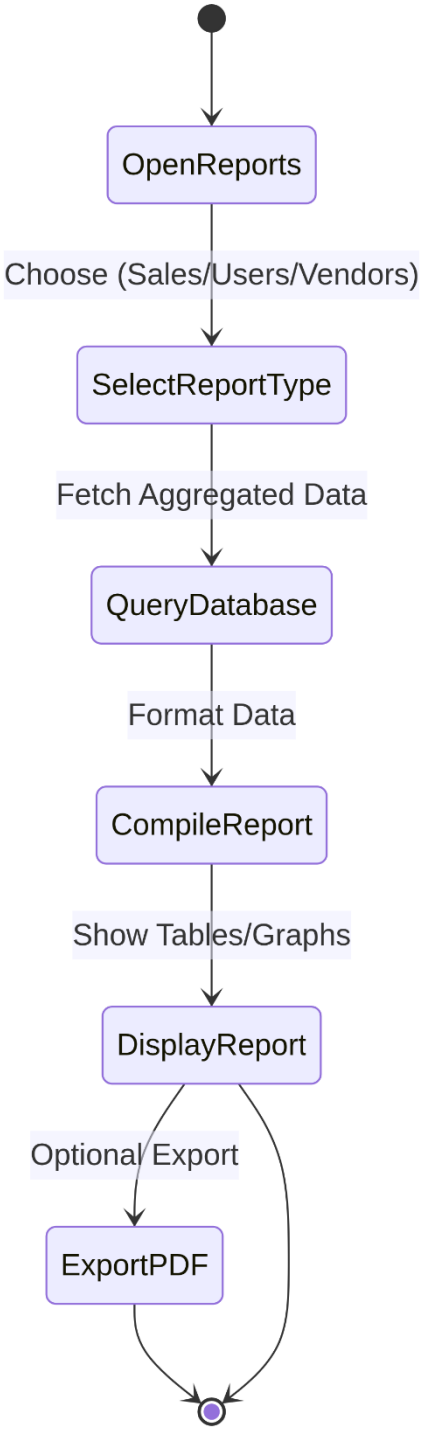
2.1.12 Configure System (Admin)

The administrator uses the System Configuration panel to manage global parameters. This includes setting campus-wide operating hours, enabling or disabling specific payment gateways, and managing system-wide alerts or maintenance windows. These settings ensure the platform operates according to university policies and technical requirements.



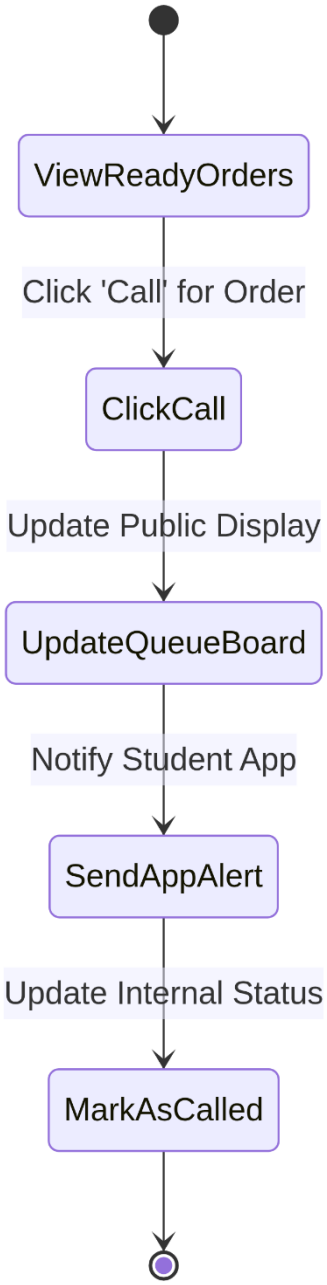
2.1.13 View Reports & Analytics (Admin)

The administrator generates high-level reports to monitor the health of the campus dining ecosystem. Reports include user registration trends, vendor performance metrics, and total transaction volumes. Data can be filtered by date, faculty, or vendor category, and the results can be exported for administrative review or compliance auditing.



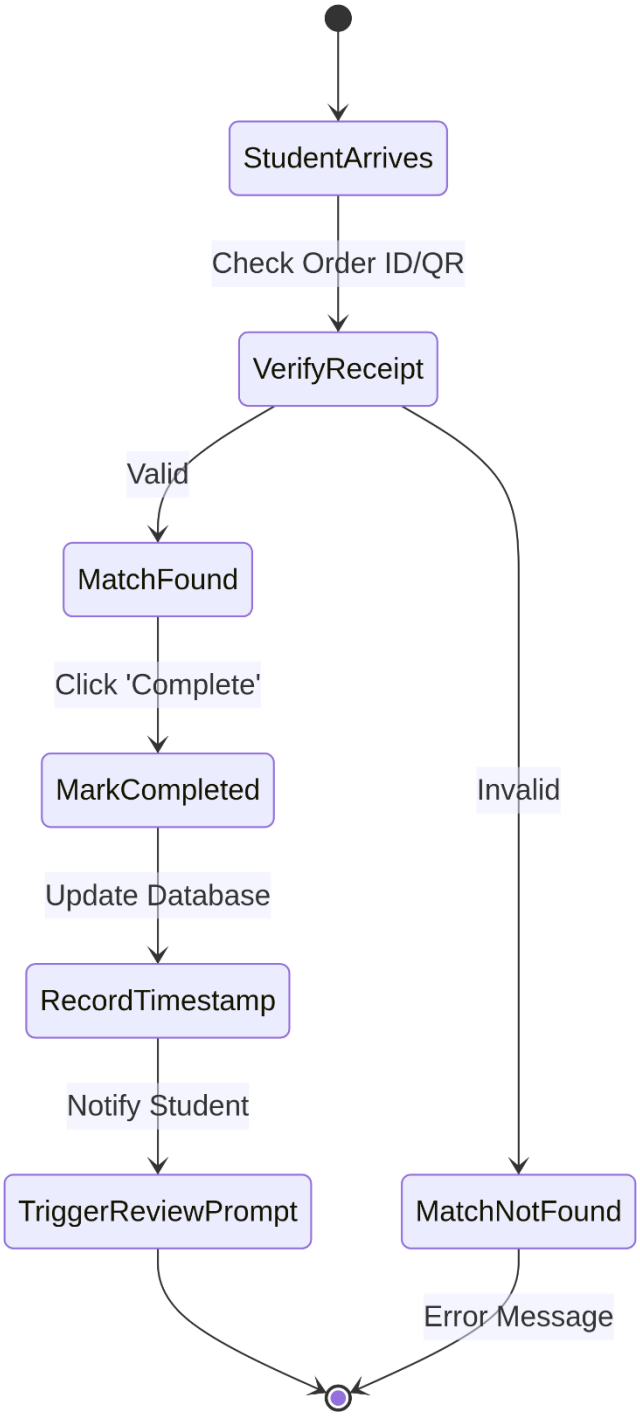
2.1.14 Call Queue Number (Staff)

When an order is marked as "Ready," the staff member uses the Call Queue function to alert the student. This action updates the public-facing queue display board and sends a high-priority notification to the student. The system tracks the time between the "Ready" status and the "Call" action to monitor service efficiency.



2.1.15 Confirm Pickup / Delivery (Staff)

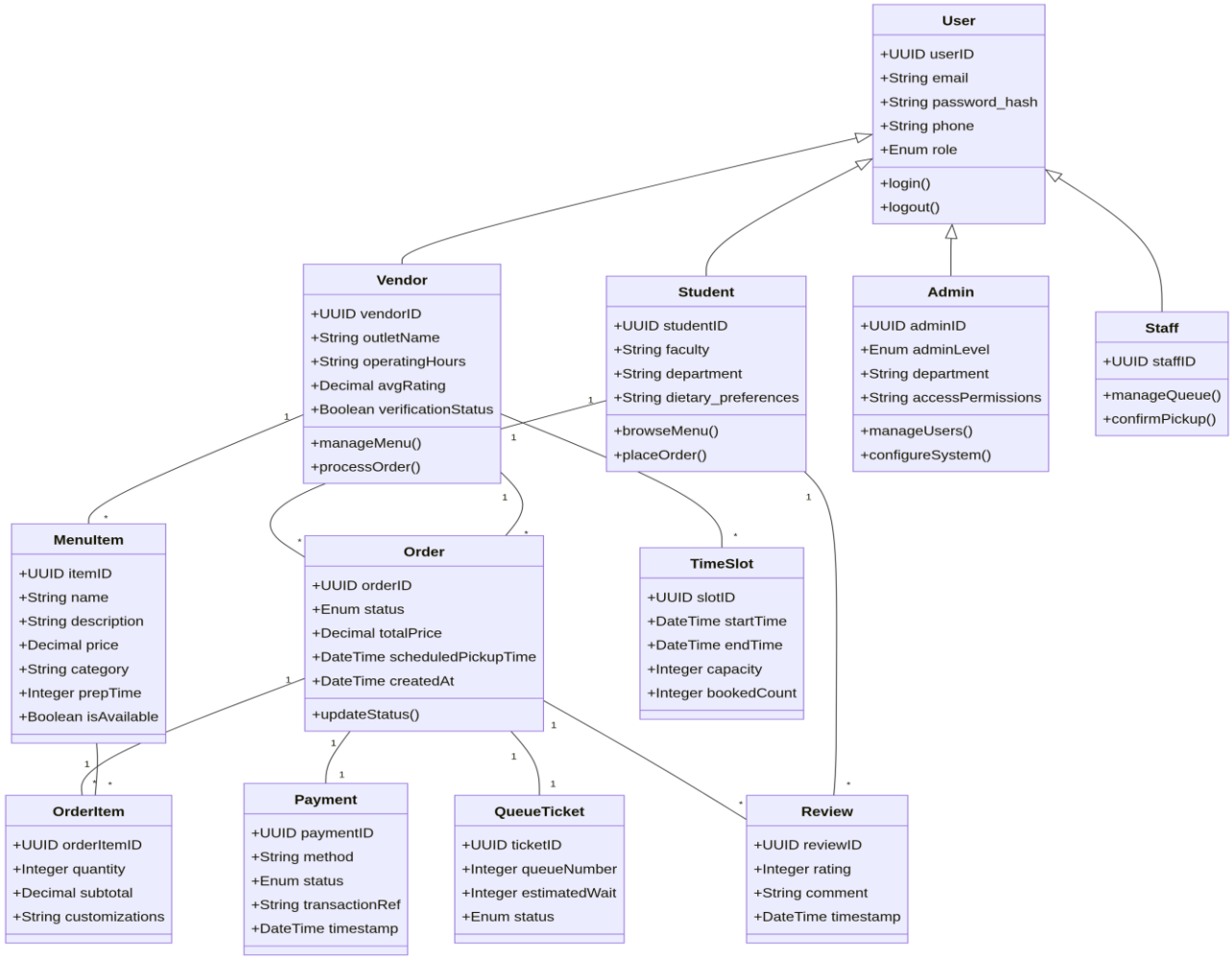
Upon the student's arrival, the staff member verifies the order receipt (e.g., via a QR code or Order ID). Once verified, the staff member marks the order as "Completed." This finalizes the transaction, records the completion timestamp for analytics, and triggers a prompt for the student to rate their experience.



3 Data Design

3.1 Design Class Diagram

The class diagram below shows the main entities of the Campus Food Ordering System, their attributes, key methods, and the associations between them. The User class serves as the base from which Student, Vendor, and Admin are derived through inheritance. The Order entity is the central transactional object, linked to Student, Vendor, OrderItem, Payment, QueueTicket, and Review.



3.2 Data Dictionary

The data dictionary below provides a comprehensive description of every attribute for each entity in the system. Data types, constraints, and descriptions are provided to guide database implementation.

Entity	Attribute	Type	Description
User	userID	UUID	Primary key. Unique identifier for every user account.
	email	String	User's email address, used as the login identifier. Must be unique.
	password_hash	String	Bcrypt-hashed password stored for secure authentication.
	phone	String	User's contact phone number.
	role	Enum	User role: Student, Vendor, Admin, or Staff.
Student	studentID	UUID	Primary key for the student entity.
	userID	UUID (FK)	Foreign key referencing User.userID.
	faculty	String	Faculty the student is enrolled in.
	department	String	Department within the faculty.
	dietary_preferences	String	Optional dietary notes (e.g., halal, vegetarian).
Vendor	vendorID	UUID	Primary key for the vendor entity.
	userID	UUID (FK)	Foreign key referencing User.userID.
	outletName	String	Name of the food outlet displayed to students.
	operatingHours	String	Vendor operating schedule (e.g., 08:00–18:00).
	avgRating	Decimal	Average rating computed from all submitted reviews (0.0–5.0).
	verificationStatus	Boolean	Admin-verified status flag for the vendor account.
Admin	adminID	UUID	Primary key for the admin entity.
	userID	UUID (FK)	Foreign key referencing User.userID.
	adminLevel	Enum	Privilege tier: SuperAdmin or RegularAdmin.
	department	String	Administrative department of the admin.

Entity	Attribute	Type	Description
	accessPermissions	String	JSON/comma-delimited list of granted permissions.
MenuItem	itemID	UUID	Primary key for the menu item.
	vendorID	UUID (FK)	Foreign key referencing Vendor.vendorID.
	name	String	Display name of the food or drink item.
	description	String	Brief description of the item.
	price	Decimal	Selling price in Malaysian Ringgit (MYR).
	category	String	Category such as Meal, Snack, Beverage, Dessert.
	prepTime	Integer	Estimated preparation time in minutes.
	isAvailable	Boolean	Availability flag; false hides item from student view.
Order	orderID	UUID	Primary key for the order.
	studentID	UUID (FK)	Foreign key referencing Student.studentID.
	vendorID	UUID (FK)	Foreign key referencing Vendor.vendorID.
	status	Enum	Order lifecycle status: Pending, Confirmed, Preparing, Ready, Completed, Cancelled.
	totalPrice	Decimal	Total payable amount including all order items.
	scheduledPickupTime	DateTime	Pre-selected pickup time for scheduled orders.
	createdAt	DateTime	Timestamp when the order was created.
	updatedAt	DateTime	Timestamp of the most recent status update.
OrderItem	orderItemID	UUID	Primary key for an order line item.
	orderID	UUID (FK)	Foreign key referencing Order.orderID.
	itemID	UUID (FK)	Foreign key referencing MenuItem.itemID.
	quantity	Integer	Number of units of the menu item ordered.
	subtotal	Decimal	price × quantity for this line item.
	customizations	String	Optional free-text special instructions for the item.
Payment	paymentID	UUID	Primary key for the payment record.

Entity	Attribute	Type	Description
	orderID	UUID (FK)	Foreign key referencing Order.orderID (1-to-1).
	method	String	Payment method: TnG, Credit Card, Debit Card, Online Banking.
	status	Enum	Payment status: Pending, Success, Failed, Refunded.
	transactionRef	String	External transaction reference from the payment gateway.
	timestamp	DateTime	Date and time the payment was processed.
QueueTicket	ticketID	UUID	Primary key for the queue ticket.
	orderID	UUID (FK)	Foreign key referencing Order.orderID (1-to-1).
	queueNumber	Integer	Sequential queue number assigned to the order.
	estimatedWait	Integer	Estimated wait time in minutes at the time of assignment.
	status	Enum	Ticket status: Waiting, Called, Served.
Review	reviewID	UUID	Primary key for the review.
	orderID	UUID (FK)	Foreign key referencing Order.orderID.
	studentID	UUID (FK)	Foreign key referencing Student.studentID.
	rating	Integer	Numeric rating from 1 (worst) to 5 (best).
	comment	String	Optional written feedback from the student.
	timestamp	DateTime	Date and time the review was submitted.
TimeSlot	slotID	UUID	Primary key for the time slot.
	vendorID	UUID (FK)	Foreign key referencing Vendor.vendorID.
	startTime	DateTime	Start of the scheduling window.
	endTime	DateTime	End of the scheduling window.
	capacity	Integer	Maximum number of orders allowed in this slot.
	bookedCount	Integer	Current number of orders booked for this slot.

3.3 Data Structures

3.3.1 Order Status Enumeration

The Order entity uses a lifecycle enumeration to track the state of each order. The permitted transitions enforce business rules (e.g., an order cannot move from Completed back to Preparing).

Status Value	Triggered By	Description
Pending	Student (payment submitted)	Order created; awaiting payment confirmation.
Confirmed	Payment Gateway (success)	Payment processed; order awaiting vendor acceptance.
Preparing	Vendor (accept order)	Vendor has accepted and is preparing the order.
Ready	Vendor (mark ready)	Order prepared; student notified to collect.
Completed	Staff (confirm pickup)	Student has collected the order; process finalized.
Cancelled	Vendor (reject) / System (timeout)	Order rejected or payment timed out; refund triggered if applicable.

3.3.2 Payment Method and Status Enumerations

The Payment entity uses two enumerations to track how and whether a payment was processed.

Enumeration	Values	Description
Payment Method	TnG, CreditCard, DebitCard, OnlineBanking	Represents the payment channel used by the student.
Payment Status	Pending, Success, Failed, Refunded	Tracks the current result of the transaction with the payment gateway.

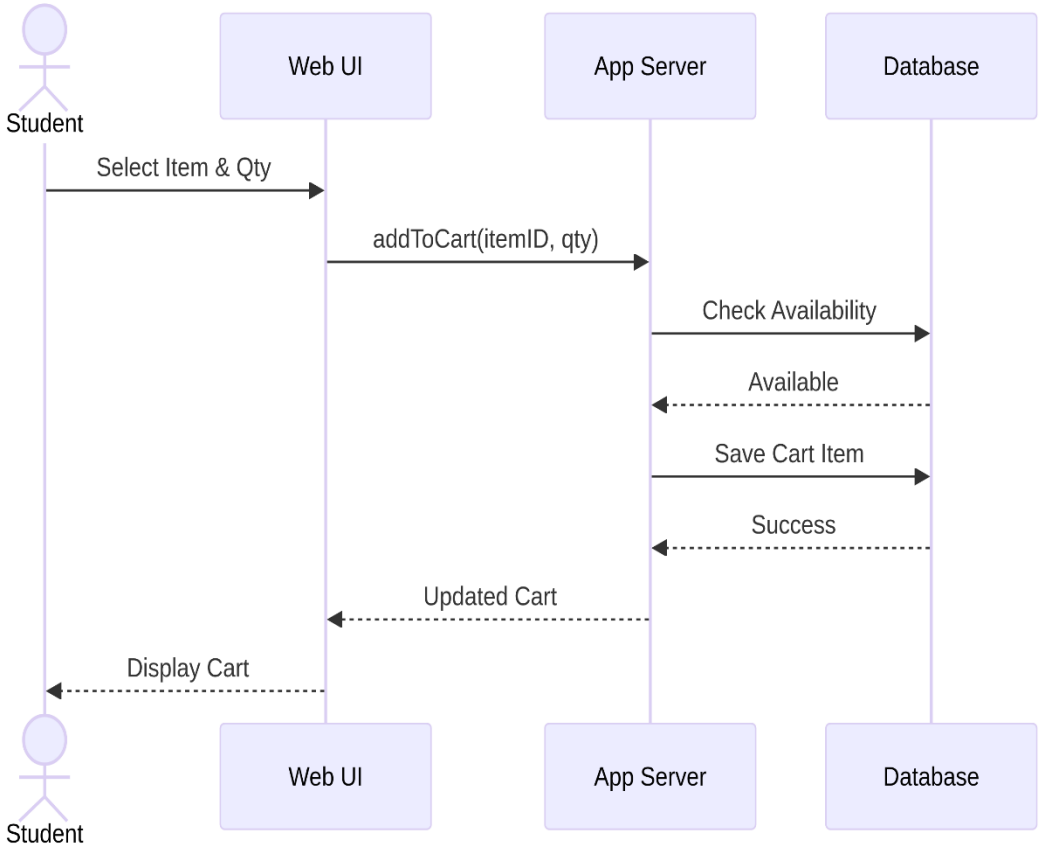
4 Behavioral Modeling

This section presents sequence diagrams that describe the dynamic interactions between actors and system components for key use cases, along with a state diagram capturing the lifecycle of the core Order entity.

4.1 Sequence Diagrams

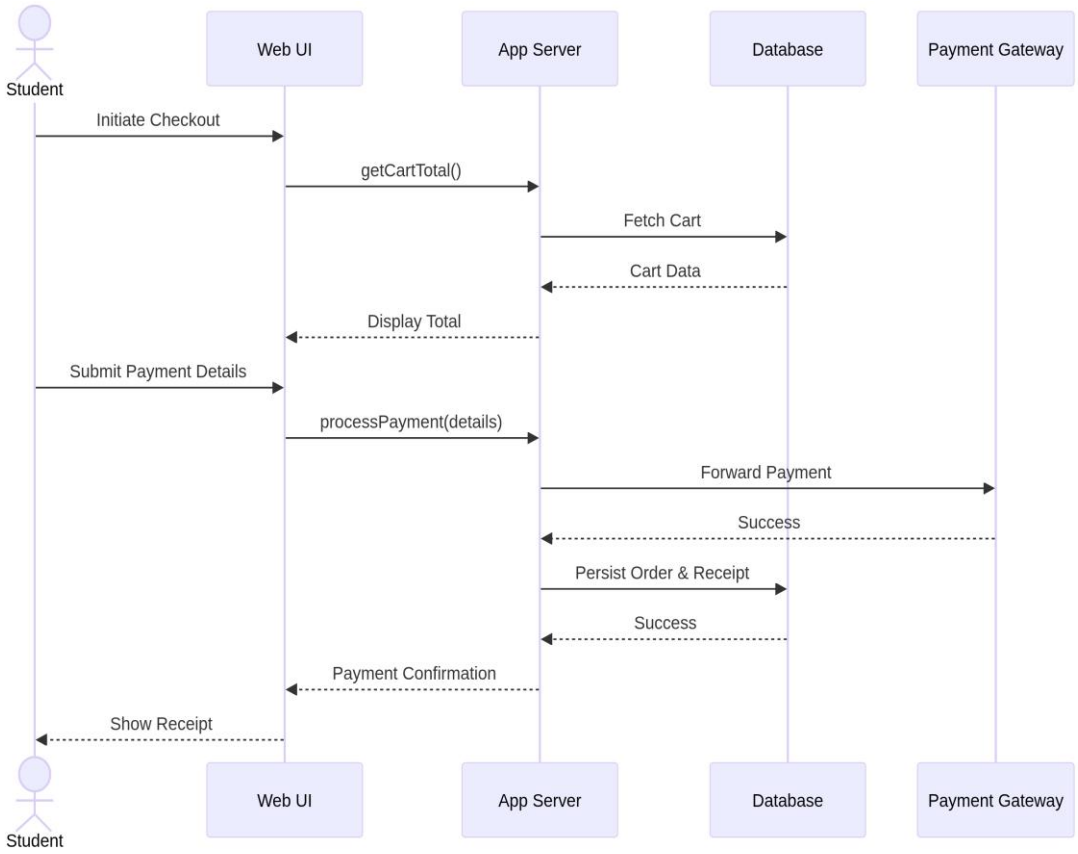
4.1.1 Add to Cart

The Student selects a food item in the Web UI, which sends an addToCart request to the App Server. The server queries the Database to verify availability. If available, the cart item is saved and the updated cart is returned to the UI. If unavailable, an error is propagated back to the student.



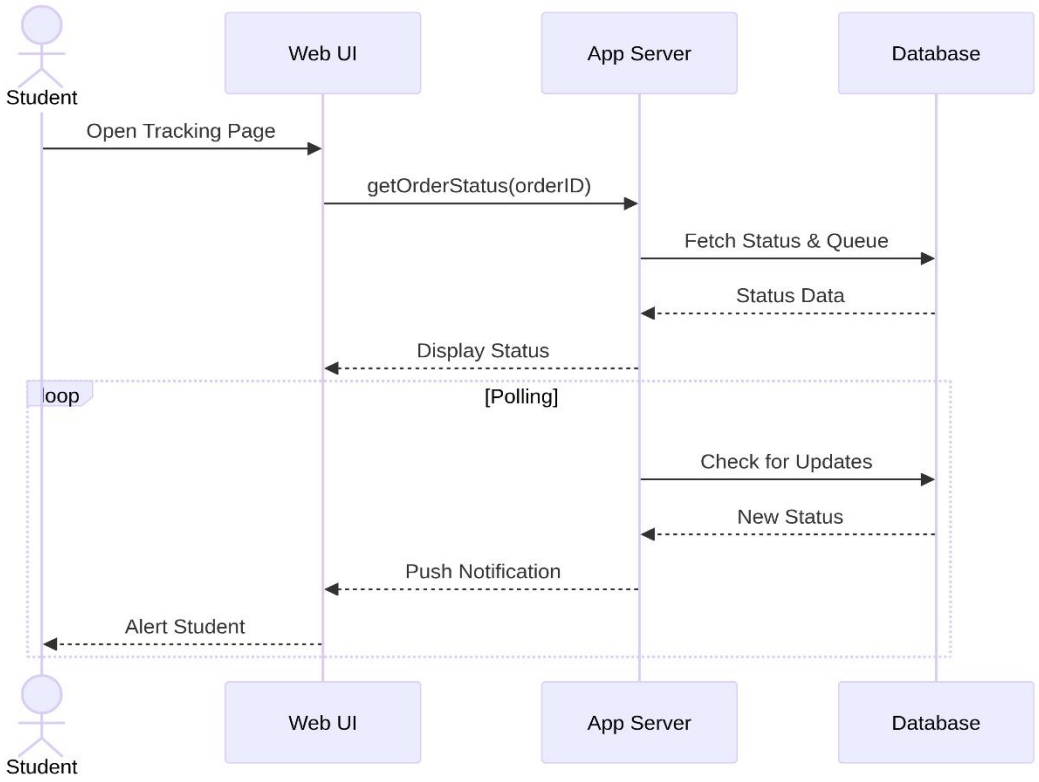
4.1.2 Make Payment

The Student initiates checkout. The App Server retrieves the cart total from the Database, displays it to the student, then forwards payment details to the external Payment Gateway. The gateway returns a success/failure result. On success, the order is persisted in the Database and a confirmation with receipt is returned to the student.



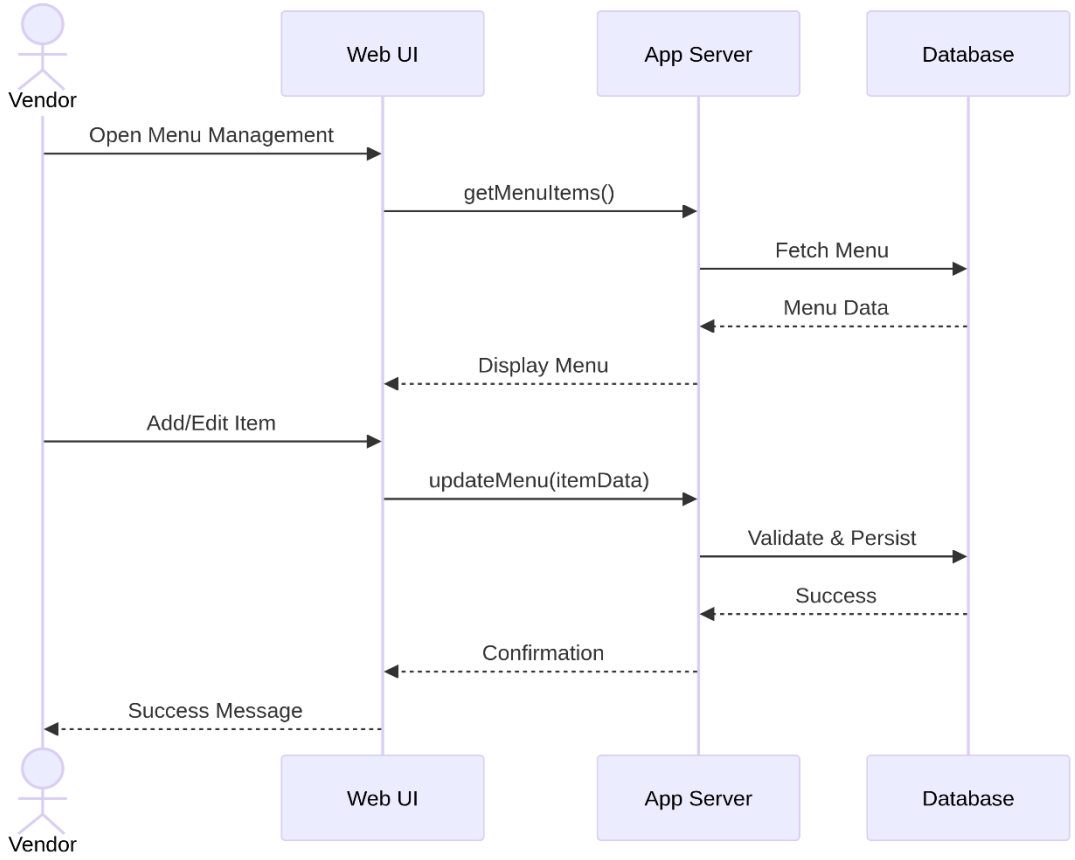
4.1.3 Track Order

The Student opens the Order Tracking page. The App Server fetches the current order status and queue position from the Database and displays it. The server then enters a polling loop; whenever the status changes, a push notification is sent through the App Server to the Web UI, which alerts the student.



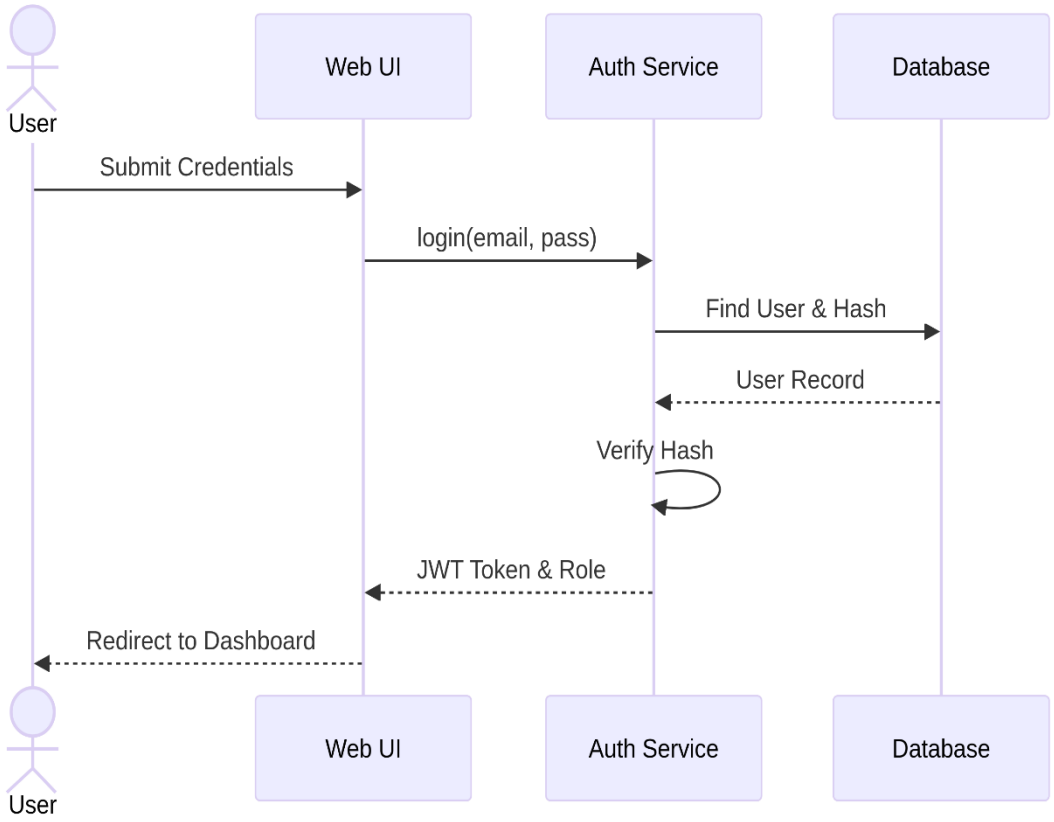
4.1.4 Manage Menu & Inventory

The Vendor opens the Menu Management page. The App Server retrieves existing menu items from the Database and displays them. The Vendor adds or edits an item; the App Server validates the input and persists the changes to the Database. A confirmation is returned to the Vendor, and the updated menu is immediately visible to students.



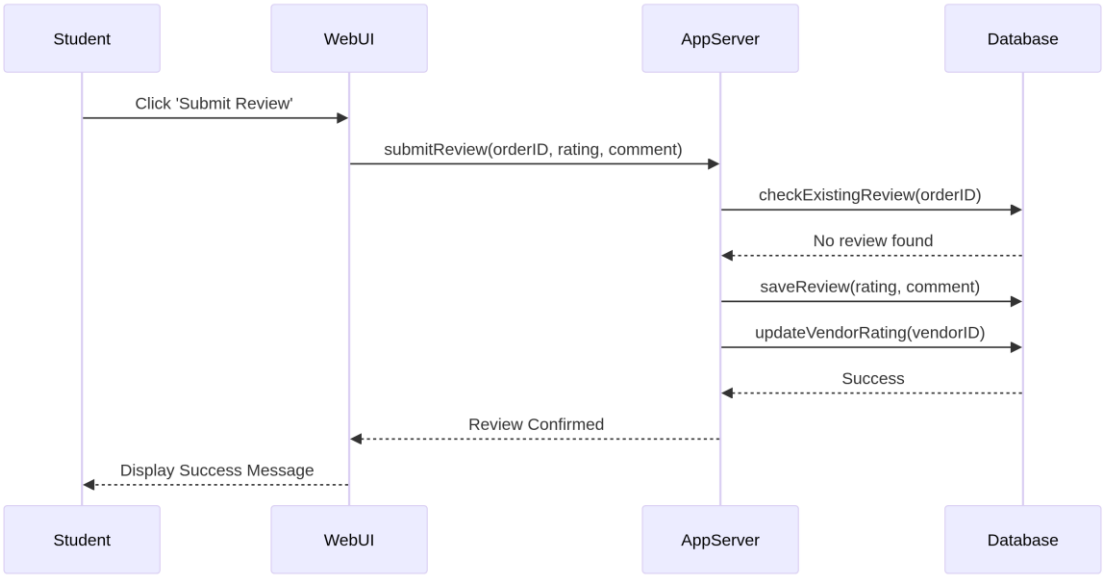
4.1.5 Login / Register

The User submits credentials (or registration details) through the Web UI. The Auth Service queries the Database to find the user record and verifies the password hash. On valid credentials, a JWT session token with the user's role is issued and the user is redirected to their dashboard. On invalid credentials, an error is returned.



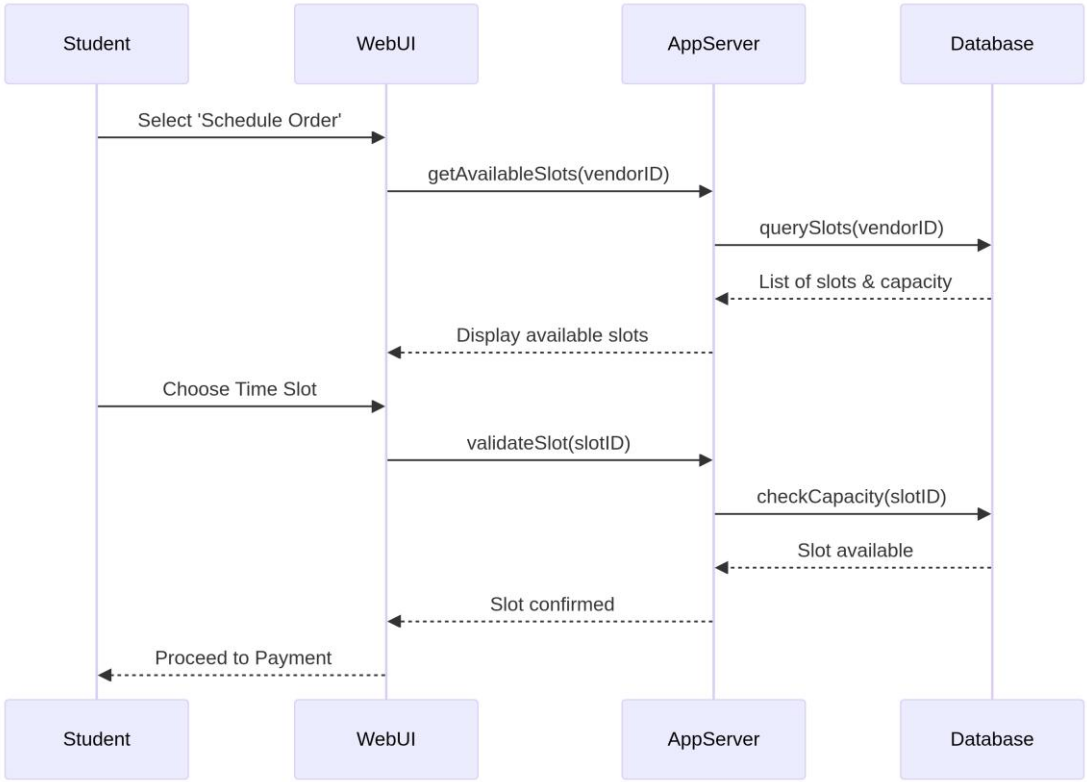
4.1.6 Submit Review

The Student submits a review for a completed order. The App Server checks the Database to ensure no duplicate review exists, then saves the rating and comment. Finally, the App Server triggers an update to the Vendor's average rating in the Database.



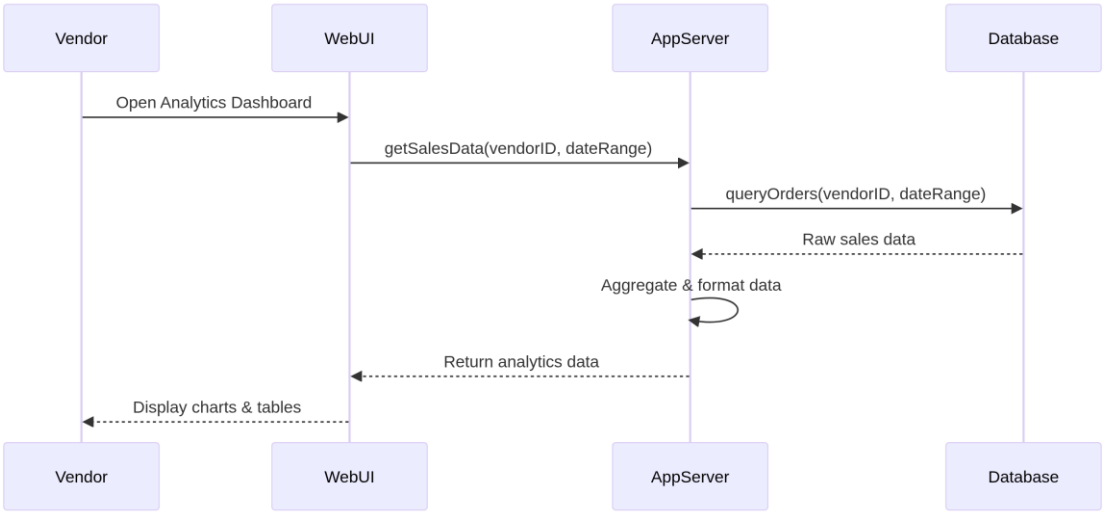
4.1.7 Schedule Order

The Student requests available time slots for a vendor. The App Server retrieves slot data and capacity from the Database. After the student chooses a slot, the server validates its availability before allowing the student to proceed to payment.



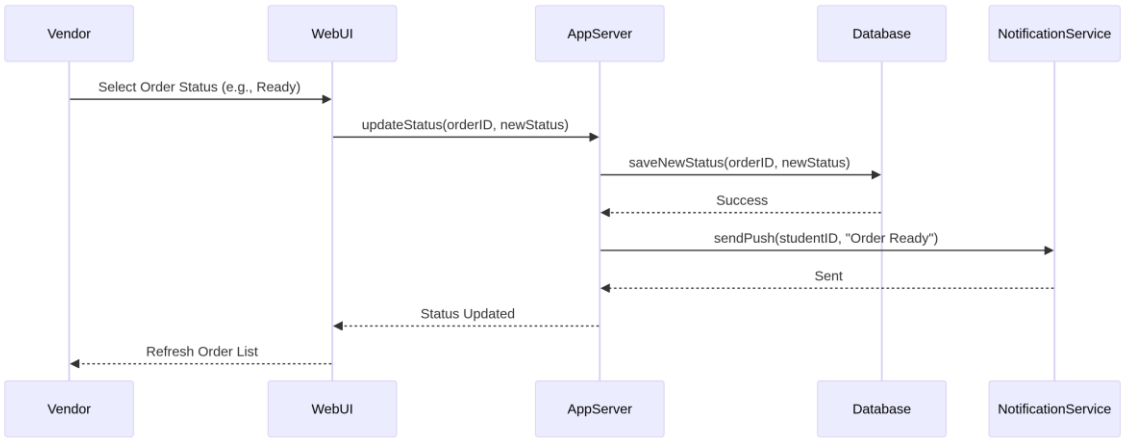
4.1.8 View Sales Analytics

The Vendor requests sales data for a specific date range. The App Server queries the Database for relevant order records, aggregates the results (e.g., total revenue, top items), and returns the formatted data to the Web UI for display.



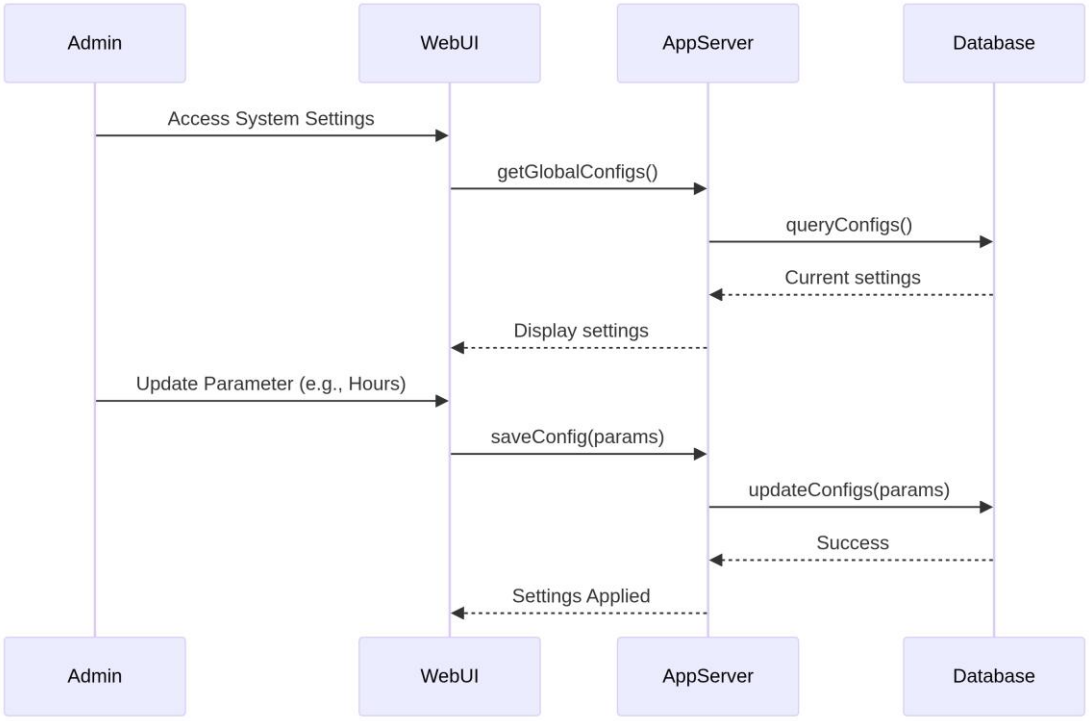
4.1.9 Update Order Status

The Vendor updates an order's status (e.g., to "Ready"). The App Server saves the new status in the Database and simultaneously triggers a push notification via the Notification Service to alert the student.



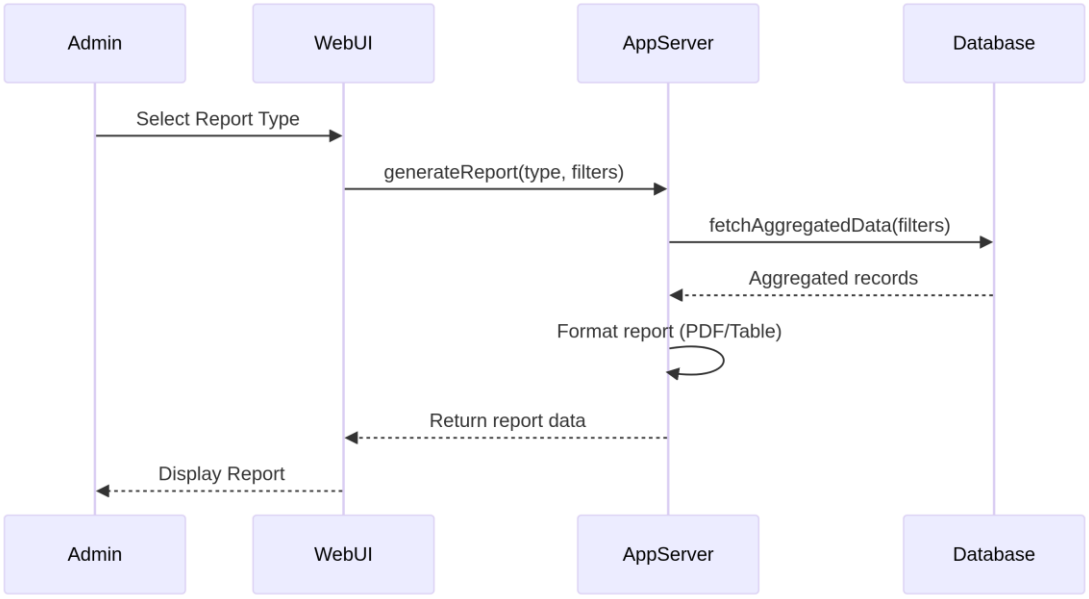
4.1.10 Configure System

The Admin accesses global system settings. The App Server fetches current configurations from the Database. When the Admin submits changes, the server updates the Database and applies the new settings across the system.



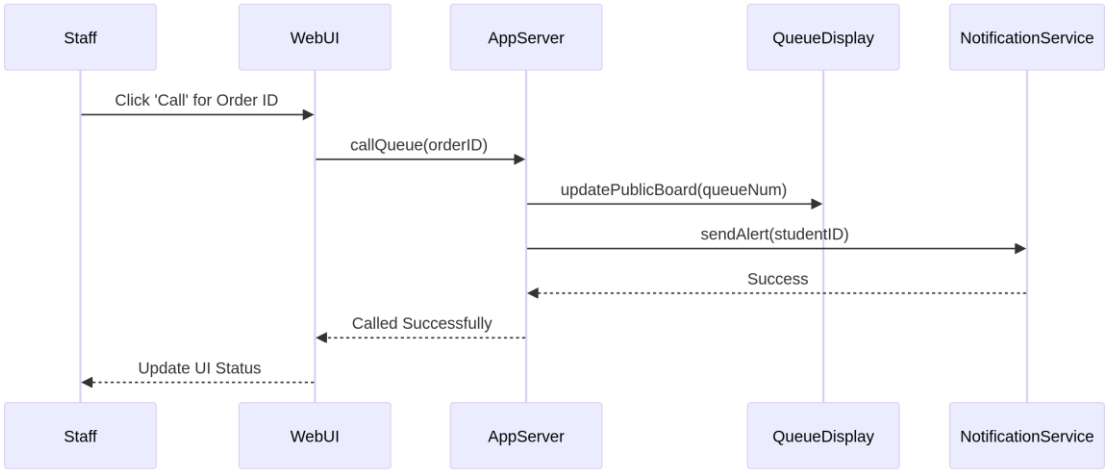
4.1.11 View Reports & Analytics

The Admin requests a high-level report. The App Server fetches aggregated data from the Database based on filters (e.g., date, category), formats the data into a report structure, and delivers it to the Admin interface.



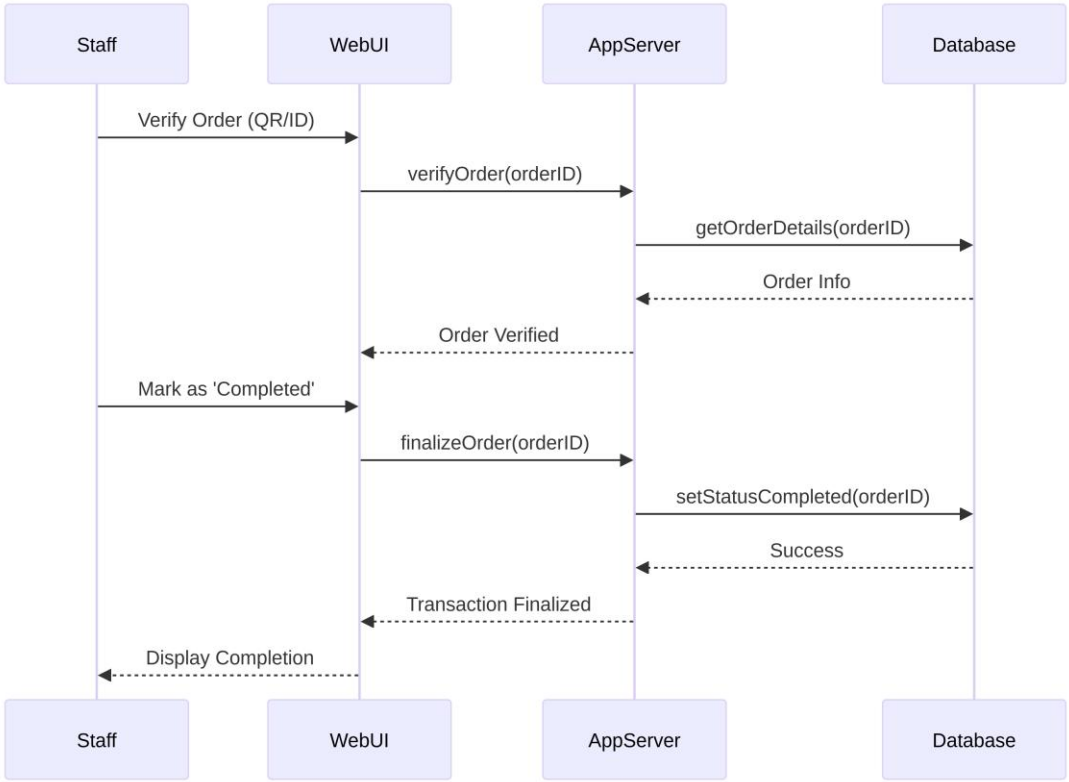
4.1.12 Call Queue Number

The Staff member triggers a "Call" action for a ready order. The App Server updates the public Queue Display board and sends a direct alert to the Student's mobile app via the Notification Service.



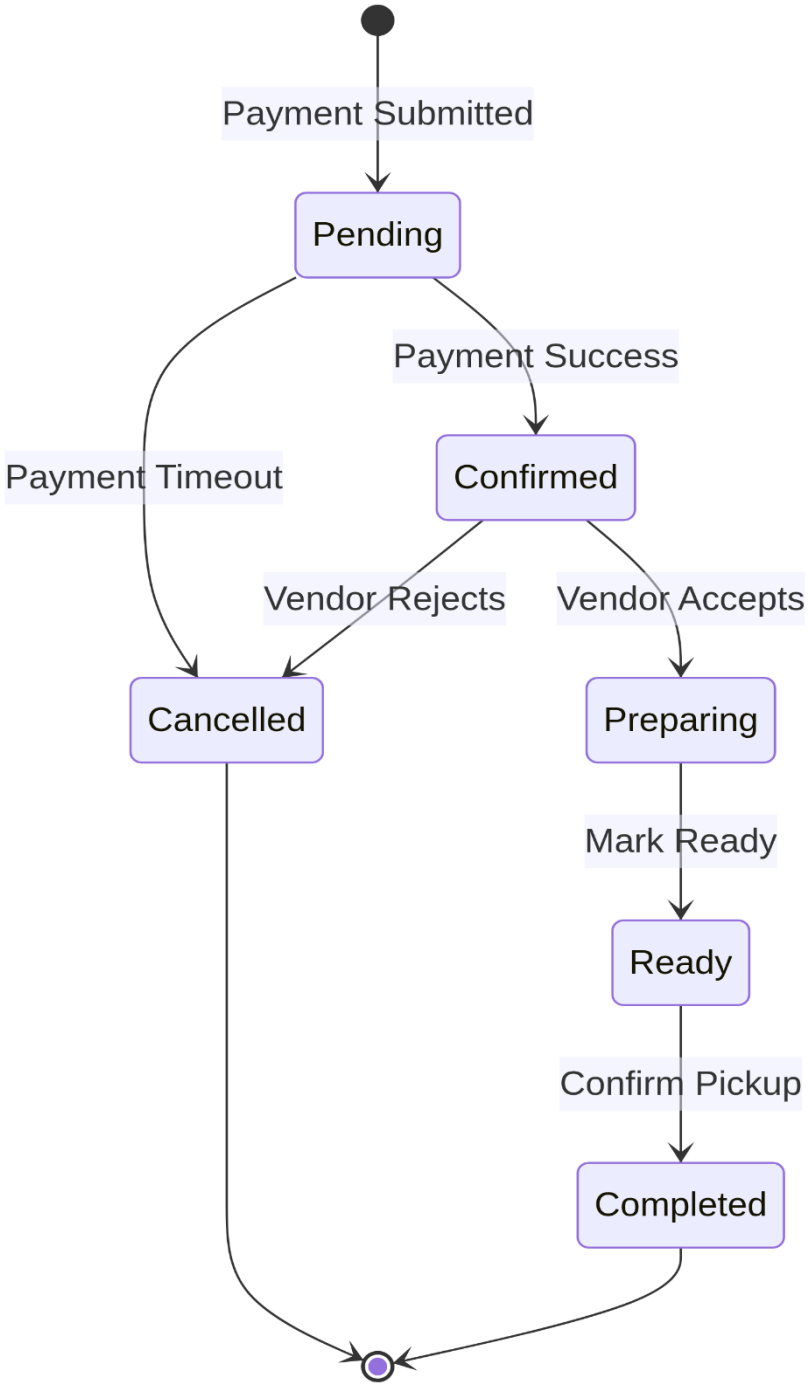
4.1.13 Confirm Pickup / Delivery

The Staff member verifies the student's order (e.g., scanning a QR code). Once verified, the staff member marks the order as "Completed" in the Web UI. The App Server updates the Database, finalizes the transaction, and confirms completion to the staff.



4.2 State Diagram

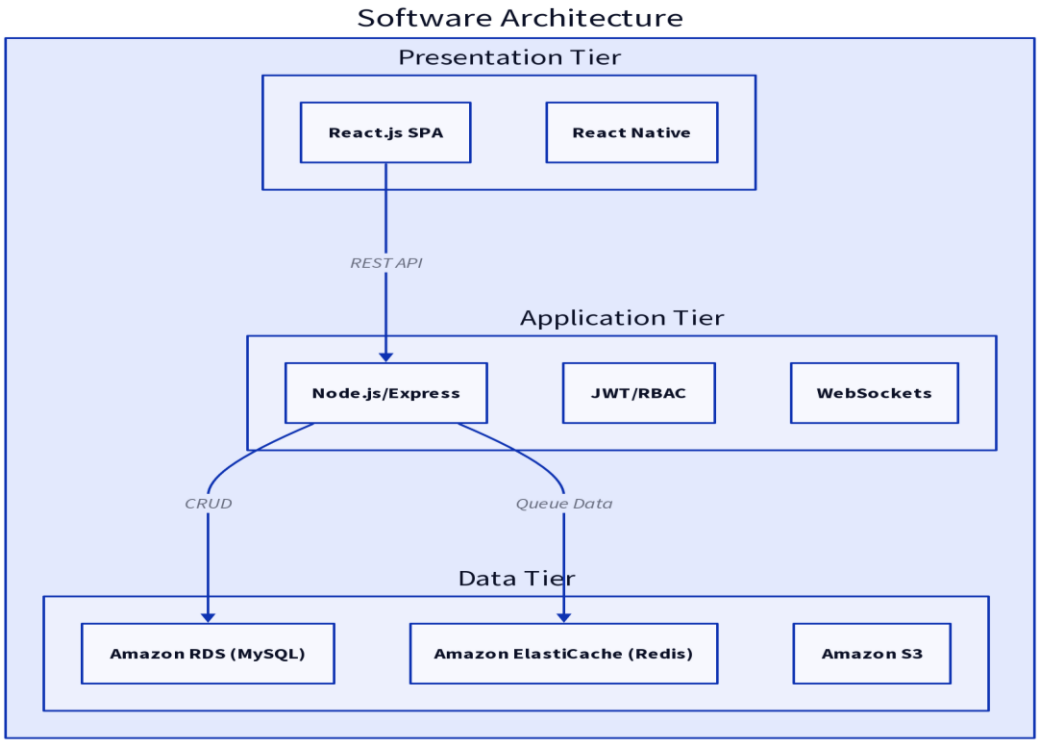
The state diagram below models the full lifecycle of an Order entity, from its creation as a Pending order through to Completed or Cancelled. State transitions are triggered by specific actor actions or system events, enforcing the business rules defined in Part I.



5 Architecture Design

5.1 Software Architecture

The Campus Food Ordering and Management System adopts a three-tier client-server architecture deployed on the cloud (AWS). This approach cleanly separates concerns into Presentation, Application, and Data tiers, enabling independent scaling, maintenance, and testing of each layer.

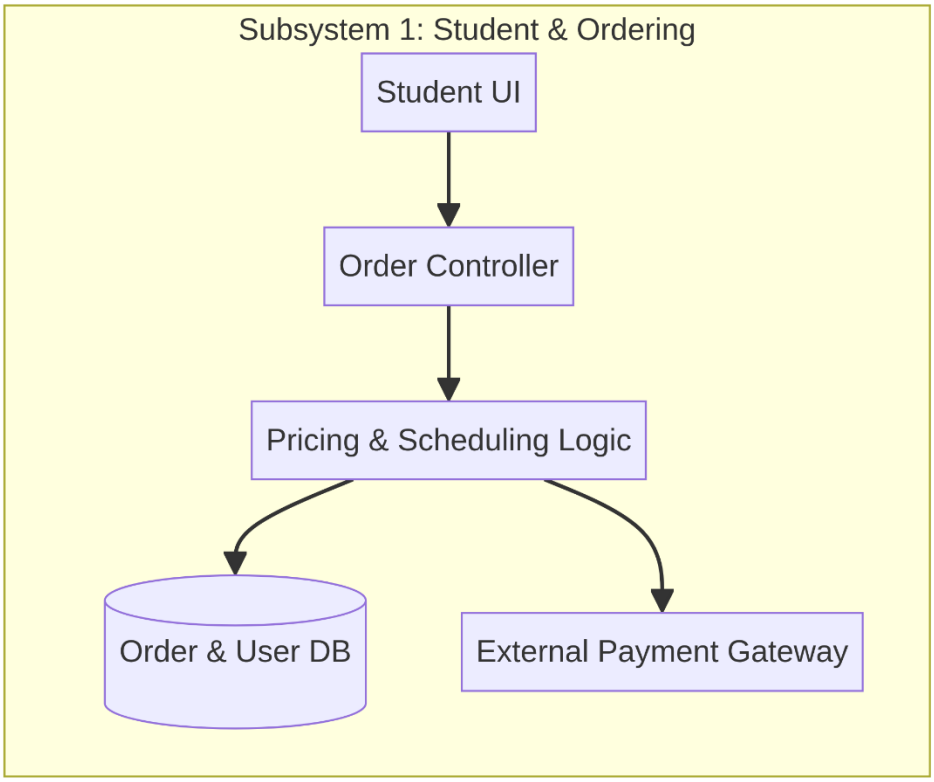


The system is divided into the following subsystems, each assigned to a team member for implementation:

- Student Subsystem (Hassan) – handles menu browsing, cart management, order scheduling, payment, order tracking, and reviews.
- Vendor Subsystem (Rashad) – handles menu & inventory management, order queue management, status updates, and sales analytics.
- Admin Subsystem (Ahmad) – handles user/vendor account management, system configuration, and report generation.
- Staff & Queue Subsystem (Mohammed) – handles queue calling, pickup/delivery confirmation, and order status updates.

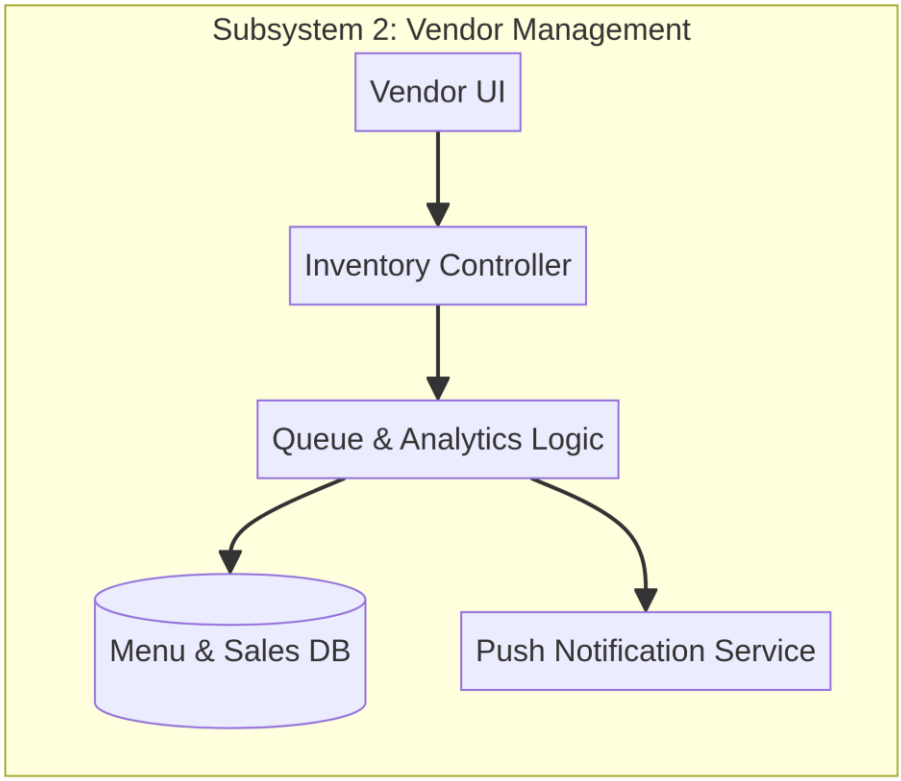
5.1.1 Subsystem 1 – Student & Ordering

This subsystem handles all student-facing features. The frontend is a React.js single-page application communicating with the backend via RESTful APIs. Key modules include the MenuController (browsing), CartService (cart management), OrderController (order placement and scheduling), PaymentService (checkout), TrackingController (real-time status), and ReviewService (post-order feedback). WebSockets are used for real-time order status and queue position updates.



5.1.2 Subsystem 2 – Vendor Management

The Vendor subsystem provides a dashboard for food outlet operators. It includes the MenuRepository (menu CRUD, CSV import, availability toggles), QueueManager (incoming order queue, accept/reject, real-time updates), OrderStatusController (Preparing/Ready updates pushed to students), and AnalyticsEngine (sales charts, revenue reports, filtered by date range). The vendor UI auto-refreshes every 10 seconds and uses WebSockets for live queue changes.



6 Interface Design

6.1 Main Screens

The system's interface is designed to be mobile-responsive, adhering to WCAG 2.1 AA accessibility standards. Navigation is role-based: students see ordering features, vendors see management tools, and admins see administrative panels. The table below summarizes all major screens of the system.

Screen Name	Actor	Key Elements & Description
Login / Register Page	All	Email and password fields, Register/Login toggle, role selection (Student/Vendor/Staff). Error messages below fields. "Forgot Password" link. Campus logo at top.
Student Home / Menu Browse	Student	Top search bar, filter by vendor/category, grid of food items with name, price, availability badge. Cart icon with item count in navbar. Banner showing active promotions.
Shopping Cart	Student	List of added items with quantity spinners, subtotal per item, grand total at bottom. "Proceed to Checkout" and "Continue Shopping" buttons. Option to remove items.
Checkout & Payment	Student	Order summary, payment method selector (TnG/Card), payment detail fields, place order button. Security badge displayed.

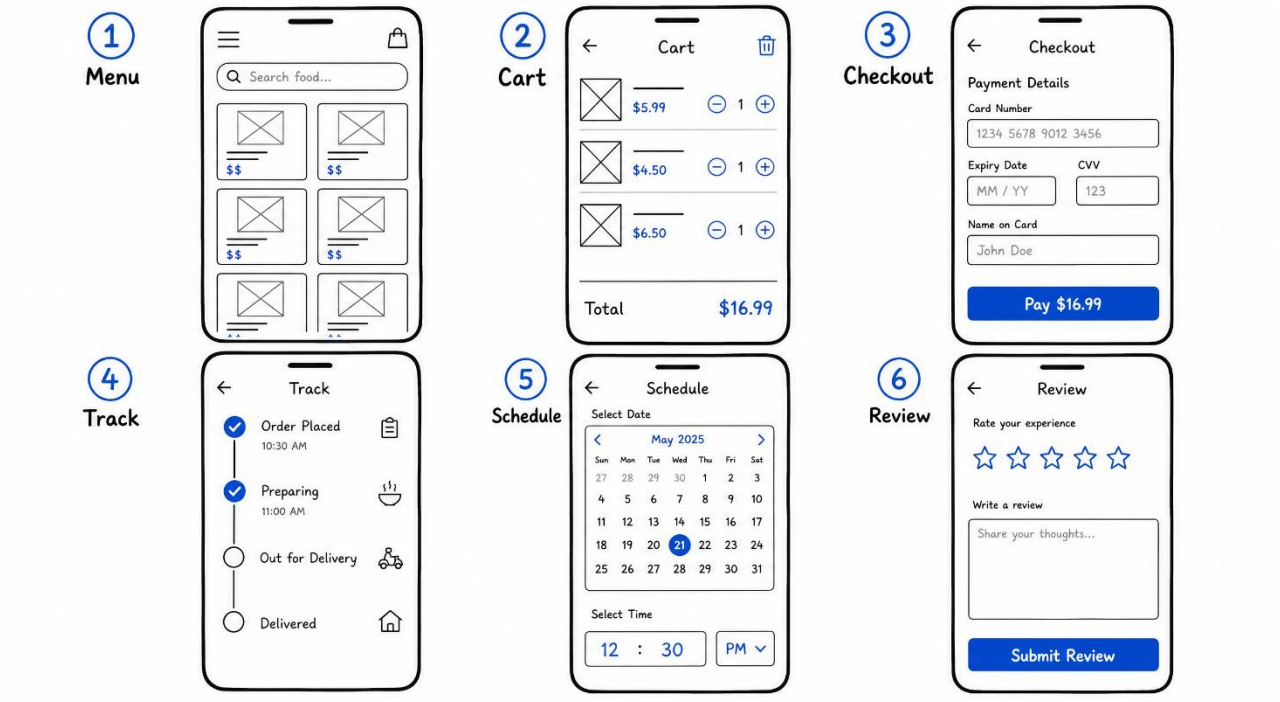
Screen Name	Actor	Key Elements & Description
Order Tracking	Student	Real-time status stepper (Pending → Confirmed → Preparing → Ready → Completed). Queue number display, estimated wait time, notification toggle.
Schedule Order	Student	Calendar date picker, time slot grid showing availability. Selected slot highlighted. Confirmation button. Already-booked slots grayed out.
Submit Review	Student	Star rating (1–5), text area for comments, submit button. Previous reviews shown below. Only visible for completed orders.
Vendor Dashboard	Vendor	Overview cards (today's orders, revenue, ratings). Order queue list with Accept/Reject buttons. Quick menu availability toggles. Navigation to Analytics and Menu Management.

Screen Name	Actor	Key Elements & Description
Manage Menu & Inventory	Vendor	Table of menu items with columns: Name, Category, Price, Stock, Status (toggle). Add New Item button opens a modal with form fields. CSV import button.
Order Queue Management	Vendor	Live list of incoming orders sorted by time. Color-coded by status. Drag-to-reorder support. Bulk accept button. Search by order ID or student name.
Admin Dashboard	Admin	System health metrics (active users, total orders today, flagged issues). Sidebar for User Management, Vendor Management, System Config, Reports.
Manage Users & Vendors	Admin	Searchable/filterable table of accounts (Name, Role, Status, Last Login). Approve/Suspend/Delete action buttons. Modal for editing account details.
Reports & Analytics	Admin	Date range filter, tabs for Sales / Users / Vendors. Bar/line charts, summary statistics, export to CSV/PDF buttons.
Staff Queue Panel	Staff	Large display of current queue number. Next/Recall buttons. List of waiting tickets. Confirm Pickup button for ready orders.

6.2 Subsystem 1 Screens (Student)

The Student subsystem includes the following primary screens: (1) Home/Menu Browse — a responsive grid of food items with search and filter functionality; (2) Cart — a summary of selected items with quantity controls and total price; (3) Checkout/Payment — a secure payment form with method selection; (4) Order Tracking — a real-time stepper showing order progress and estimated wait time; (5) Schedule Order — a date/time picker for pre-ordering; (6) Submit Review — a star-rating form for completed orders.

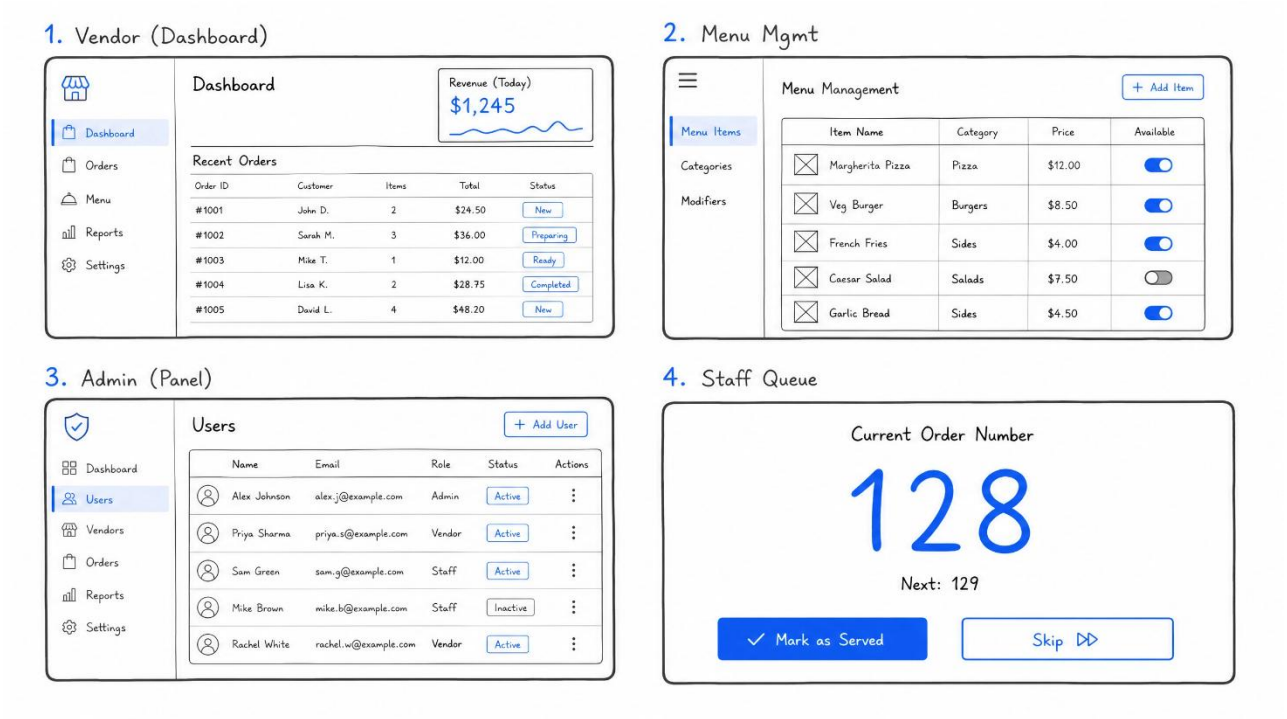
All screens use a consistent color scheme (blue primary, white background), a fixed top navigation bar with cart badge, and contextual toast notifications for system events (order confirmed, payment successful, order ready).



6.3 Subsystem 2 Screens (Vendor, Admin & Staff)

The Vendor Dashboard provides a summary of today's orders and revenue at a glance. The Order Queue panel is a live-updating list with color-coded status badges and Accept/Reject action buttons. The Menu Management screen uses an editable table with inline toggles for item availability.

The Admin Panel uses a sidebar navigation pattern for User Management, Vendor Management, System Configuration, and Reports. The Staff Queue Panel is optimized for display on a countertop screen, featuring a large current queue number and a Next/Recall control.



7 Component Design

7.1 Main Components

The system is organized into loosely coupled, single-responsibility components across the backend. Each component maps to one or more use cases and belongs to a defined subsystem. The table below lists all major components, their subsystem, and responsibilities.

Component	Subsystem	Responsibility
AuthController	User Management	Handles login, registration, JWT token issuance, and role-based access enforcement.
OrderController	Order Management	Processes Add-to-Cart, checkout, order status transitions, and scheduled orders.
PaymentService	Payment Processing	Interfaces with external payment gateway (TnG/Stripe), validates transactions, and records receipts.
QueueManager	Queue Management	Assigns queue numbers, manages order position, pushes real-time updates via WebSocket.
MenuRepository	Vendor Management	CRUD operations for menu items, inventory checks, and bulk CSV import.
AnalyticsEngine	Reporting	Aggregates sales data, computes vendor ratings, generates reports in PDF/CSV.
NotificationService	Notifications	Sends push notifications and email alerts when order status changes or queue numbers are called.
ReviewService	Student Features	Saves student reviews, validates order completion before allowing submission, updates vendor avgRating.
SchedulerService	Order Management	Manages time slots, validates booking capacity, triggers order confirmation at scheduled time.
AdminPanel	Administration	UI and API layer for user/vendor CRUD, system configuration, and report generation.

7.1.1 OrderController – Core Ordering Logic

The OrderController is the central component managing the order lifecycle. Its key algorithmic responsibilities are described below:

Add to Cart Pseudocode:

```
FUNCTION addToCart(studentID, itemID, quantity):
  item = MenuRepository.findByID(itemID)
  IF item.isAvailable AND quantity > 0 THEN
    CartService.addItem(studentID, item, quantity)
    RETURN updatedCart
  ELSE
    RAISE ItemUnavailableException
END FUNCTION
```

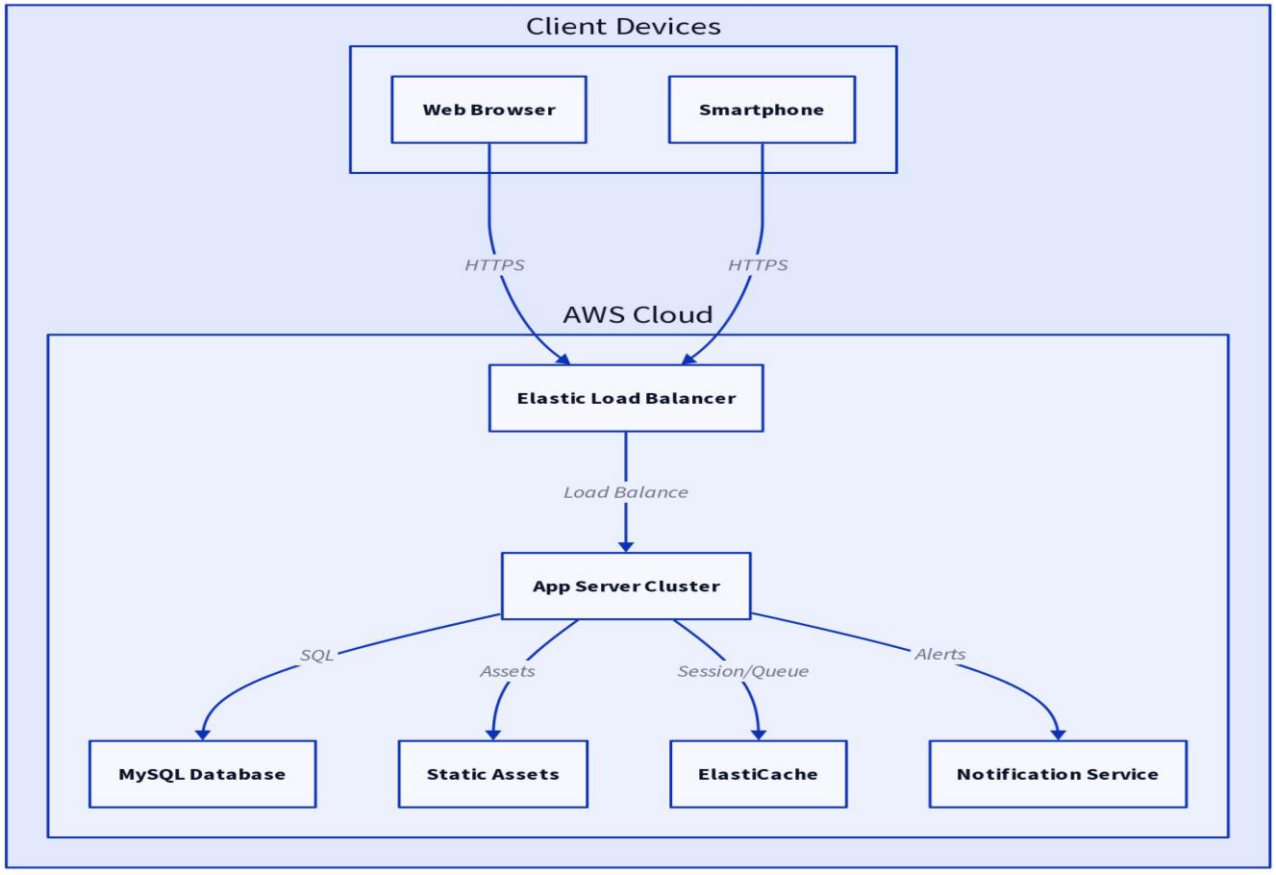
Order Status Transition Pseudocode:

```
FUNCTION updateOrderStatus(orderID, newStatus, actorRole):
  order = OrderRepository.findByID(orderID)
  IF isValidTransition(order.status, newStatus, actorRole) THEN
    order.status = newStatus
    OrderRepository.save(order)
    NotificationService.notify(order.studentID, newStatus)
  ELSE
    RAISE InvalidStatusTransitionException
END FUNCTION
```

8 Deployment Design

8.1 Deployment Diagram

The system is deployed on AWS cloud infrastructure following a microservice-friendly layered deployment. Client devices (mobile apps and web browsers) communicate with the system via HTTPS through an AWS Elastic Load Balancer. The application server (Node.js/Express) runs in an EC2 Auto-Scaling group. Persistent data is stored in Amazon RDS (MySQL/PostgreSQL), session and queue data are cached in Redis (ElastiCache), static assets and menu images are stored in Amazon S3, and notifications are dispatched via Amazon SNS. All communication between tiers uses TLS 1.3 encryption.



9 Summary

This Software Design Specification (SDS) documents the complete design of the Campus Food Ordering and Management System for Part II of the CSE6214 project. The document covers:

- Activity diagrams for seven key use cases, including Add to Cart, Make Payment, Track Order, Manage Menu, Manage Order Queue, Manage Users, and Login/Register.
- A complete Data Design, comprising the Design Class Diagram, a fully detailed Data Dictionary for all 11 entities, and structured data enumerations for Order Status and Payment.
- Behavioral Modeling via five sequence diagrams and one order lifecycle state diagram, showing the dynamic interactions between actors and system components.
- A three-tier Software Architecture design (Presentation, Application, Data) with four subsystems assigned to team members, and a cloud-based AWS Deployment Diagram.
- Interface Design descriptions for 14 screens across all four actor roles, covering both Student and Vendor/Admin/Staff subsystems.
- Component Design documentation for 10 backend components, including pseudocode for the core order logic and status transition algorithm.

The system design is aligned with the functional and non-functional requirements specified in Part I (SRS). The modular architecture and clear subsystem boundaries position the team to proceed confidently into Part III development and testing. All design decisions prioritize security (TLS 1.3, bcrypt, RBAC, PCI-DSS compliance), performance (Redis caching, WebSocket real-time updates), and usability (mobile-responsive, WCAG 2.1 AA).

References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.
- Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design (3rd ed.). Prentice Hall.
- OWASP Foundation (2024). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org
- Amazon Web Services (2024). AWS Architecture Best Practices. https://aws.amazon.com/architecture
- World Wide Web Consortium (2018). Web Content Accessibility Guidelines (WCAG) 2.1.
- Payment Card Industry Security Standards Council (2022). PCI DSS v4.0.
- diagrams.net (draw.io). Online Diagramming Tool used for UML diagrams.